



Instituto INFNET
Escola Superior da Tecnologia da Informação
EDC - Graduação em Engenharia de Computação
EST - Fundamentos da Computação

Renato Noronha Hack

Teste de Performance - AT

Programação Assembly ARM 64-bits

Mario Pego

Rio de Janeiro, 25 de março de 2026

1. 0

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q1$ qemu-aarch64 -g 2401 ./q1 &
[1] 7144
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q1$ gdb-multiarch -q ./q1
Reading symbols from ./q1...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:22
warning: Source file is more recent than executable.
22      adr x0, input_samples                // x0 = endereco do vetor de entrada
(gdb) break after_extreme_case
Breakpoint 1 at 0x400158: file main.S, line 28.
(gdb) continue
Continuing.

Breakpoint 1, after_extreme_case () at main.S:28
28      adr x9, status_primary              // x9 = endereco para salvar o status do caso principal
(gdb) print/u $w0
$1 = 0
(gdb) x/5wd &output_samples
0x41026c:      8388607 -8388608      617000 -8388608
0x41027c:      0
(gdb) continue
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) quit
[1]+  Done                  qemu-aarch64 -g 2401 ./q1
```

main.S

```
.data
input_samples:
    .hword 32767, -32768, 1234, -20000, 0
.balign 4
output_samples:
    .word 0, 0, 0, 0, 0
expected_samples:
    .word 8388607, -8388608, 617000, -8388608, 0
status_primary:
    .word 0
status_zero:
    .word 0
status_null:
    .word 0
dummy_out:
    .word 0

.text
.global _start
_start:
```

```

    adr x0, input_samples           // x0 = endereço do vetor de
entrada
    adr x1, output_samples         // x1 = endereço do vetor de
saida
    mov x2, #5                     // x2 = quantidade de
amostras
    mov w3, #500                   // w3 = ganho inteiro usado
na normalizacao
    bl normalize_saturate_i16_to_i32 // chama a rotina principal
de normalizacao
after_extreme_case:
    adr x9, status_primary         // x9 = endereço para salvar
o status do caso principal
    str w0, [x9]                   // grava o status retornado
pela funcao

    adr x0, input_samples           // x0 = mesmo vetor de
entrada para testar N=0
    adr x1, dummy_out              // x1 = saida dummy apenas
para validar status
    mov x2, #0                     // x2 = N=0 para disparar
erro especifico
    mov w3, #3                     // w3 = ganho arbitrario
para o teste
    bl normalize_saturate_i16_to_i32 // chama a rotina para
validar o retorno de N=0
after_zero_case:
    adr x9, status_zero            // x9 = endereço para salvar
o status do caso N=0
    str w0, [x9]                   // grava o status retornado

    mov x0, xzr                    // x0 = ponteiro nulo para
testar erro de ponteiro
    adr x1, dummy_out              // x1 = saida dummy para o
teste
    mov x2, #1                     // x2 = uma amostra para o
caso de ponteiro nulo
    mov w3, #1                     // w3 = ganho arbitrario
para o teste

```

```

        bl normalize_saturate_i16_to_i32           // chama a rotina para
validar erro de ponteiro
after_null_case:
        adr x9, status_null                       // x9 = endereco para salvar
o status do caso nulo
        str w0, [x9]                             // grava o status retornado

        adr x10, output_samples                  // x10 = ponteiro para a
saida gerada
        adr x11, expected_samples                // x11 = ponteiro para a
saida esperada
        mov x12, #5                             // x12 = contador de
comparacao dos 5 elementos

.check_loop:
        ldr w13, [x10], #4                       // le um valor produzido e
avanca no vetor de saida
        ldr w14, [x11], #4                       // le o valor esperado
correspondente
        cmp w13, w14                             // compara valor produzido
com valor esperado
        b.ne .fail                               // falha se houver
divergencia
        subs x12, x12, #1                        // decrementa o contador de
elementos restantes
        b.ne .check_loop                        // continua enquanto ainda
houver elementos para validar

        adr x9, status_primary                   // x9 = endereco do status
do caso principal
        ldr w10, [x9]                           // carrega o status do caso
principal
        cmp w10, #0                             // verifica se o status foi
OK
        b.ne .fail                               // falha se o status nao for
0

        adr x9, status_zero                      // x9 = endereco do status
do caso N=0

```

```

    ldr w10, [x9]                // carrega o status do caso
N=0
    cmp w10, #1                  // verifica se o retorno foi
1
    b.ne .fail                  // falha se o retorno
estiver incorreto

    adr x9, status_null          // x9 = endereco do status
do caso nulo
    ldr w10, [x9]                // carrega o status do caso
nulo
    cmp w10, #2                  // verifica se o retorno foi
2
    b.ne .fail                  // falha se o retorno
estiver incorreto

    mov w0, wzr                  // w0 = exit code 0
indicando sucesso total
    b .exit                      // encerra o programa com
sucesso

.fail:
    mov w0, #1                  // w0 = exit code 1
indicando falha em alguma validacao

.exit:
    mov x8, #93                  // x8 = syscall exit no
Linux AArch64
    svc #0                       // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.S

```

.text
.global normalize_saturate_i16_to_i32
.type normalize_saturate_i16_to_i32, %function

normalize_saturate_i16_to_i32:

```

```

    cbz x0, .null_ptr           // retorna erro se o
ponteiro de entrada for nulo
    cbz x1, .null_ptr           // retorna erro se o
ponteiro de saida for nulo
    cbz x2, .zero_len           // retorna erro se N for
zero

    mov x4, #-8388608           // x4 = limite minimo de
saturacao
    mov x5, #8388607           // x5 = limite maximo de
saturacao

.loop:
    ldrsh w6, [x0], #2          // le uma amostra int16 com
extensao de sinal e avanca a entrada
    smull x7, w6, w3            // multiplica a amostra pelo
ganho em 64 bits
    cmp x7, x4                 // compara o resultado com o
limite minimo
    b.lt .sat_min              // satura para o minimo se
estiver abaixo do intervalo
    cmp x7, x5                 // compara o resultado com o
limite maximo
    b.gt .sat_max              // satura para o maximo se
estiver acima do intervalo
    str w7, [x1], #4           // grava o valor normalizado
sem saturacao e avanca a saida
    b .next                    // segue para a proxima
iteracao

.sat_min:
    str w4, [x1], #4           // grava o limite minimo
saturado na saida
    b .next                    // segue para a proxima
iteracao

.sat_max:
    str w5, [x1], #4           // grava o limite maximo
saturado na saida

```

```

.next:
    subs x2, x2, #1                // decrementa o numero de
amostras restantes
    b.ne .loop                    // repete enquanto ainda
houver amostras

    mov w0, wzr                    // retorna status 0 para
indicar sucesso
    ret                            // volta ao chamador

.zero_len:
    mov w0, #1                    // retorna status 1 para N
igual a zero
    ret                            // volta ao chamador

.null_ptr:
    mov w0, #2                    // retorna status 2 para
ponteiro nulo
    ret                            // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

2. 0

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q2$ qemu-aarch64 -g 2401 ./q2 &
[1] 8661
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q2$ gdb-multiarch -q ./q2
Reading symbols from ./q2...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:37
37      adr x9, x_zero                      // x9 = endereco do valor x=0
(gdb) break after_zero_case
Breakpoint 1 at 0x40015c: file main.S, line 44.
(gdb) break after_unit_case
Breakpoint 2 at 0x400184: file main.S, line 56.
(gdb) continue
Continuing.

Breakpoint 1, after_zero_case () at main.S:44
44      adr x11, result_zero                // x11 = endereco para guardar o resultado do caso x=0
(gdb) print/u $w1
$1 = 0
(gdb) print (double)$d0
$2 = 0
(gdb) continue
Continuing.

Breakpoint 2, after_unit_case () at main.S:56
56      adr x11, result_one                 // x11 = endereco para guardar o resultado do caso x=1
(gdb) print/u $w1
$3 = 0
(gdb) print (double)$d0
$4 = 0.84147098480789373
(gdb) continue
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) quit
[1]+  Done                  qemu-aarch64 -g 2401 ./q2
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q2$
```

main.s

```
.data
.balign 8
x_zero:
    .double 0.0
x_one:
    .double 1.0
eps_small:
    .double 0.00000000000001
eps_zero:
    .double 0.0
expected_one:
    .double 0.8414709848078965
tol:
    .double 0.000001
result_zero:
    .double 0.0
result_one:
    .double 0.0
result_bad_eps:
    .double 0.0
result_bad_iter:
    .double 0.0
.balign 4
status_zero_case:
```



```

        .word 0
status_one_case:
        .word 0
status_bad_eps:
        .word 0
status_bad_iter:
        .word 0

.text
.global _start

_start:
    adr x9, x_zero           // x9 = endereco do valor
x=0
    ldr d0, [x9]             // d0 = argumento x do seno
    adr x10, eps_small       // x10 = endereco do epsilon
valido
    ldr d1, [x10]            // d1 = epsilon da
aproximacao
    mov w0, #8               // w0 = maximo de iteracoes
para o teste
    bl sin_taylor            // calcula sin(0) por Taylor
after_zero_case:
    adr x11, result_zero     // x11 = endereco para
guardar o resultado do caso x=0
    str d0, [x11]            // grava a aproximacao
retornada
    adr x11, status_zero_case // x11 = endereco para
guardar o status do caso x=0
    str w1, [x11]            // grava o status retornado

    adr x9, x_one           // x9 = endereco do valor
x=1.0
    ldr d0, [x9]             // d0 = argumento x do seno
    adr x10, eps_small       // x10 = endereco do epsilon
valido
    ldr d1, [x10]            // d1 = epsilon da
aproximacao
    mov w0, #12              // w0 = numero maior de
iteracoes para melhorar a precisao

```

```

        bl sin_taylor                // calcula sin(1.0) por
Taylor
after_unit_case:
        adr x11, result_one          // x11 = endereco para
guardar o resultado do caso x=1
        str d0, [x11]               // grava a aproximacao
retornada
        adr x11, status_one_case     // x11 = endereco para
guardar o status do caso x=1
        str w1, [x11]               // grava o status retornado

        adr x9, x_one                // x9 = endereco do valor
x=1.0 para testar epsilon invalido
        ldr d0, [x9]                 // d0 = argumento x do seno
        adr x10, eps_zero            // x10 = endereco do epsilon
invalido
        ldr d1, [x10]               // d1 = epsilon zero
        mov w0, #12                 // w0 = iteracoes
arbitrarias para o teste
        bl sin_taylor                // chama a funcao esperando
erro de epsilon
after_bad_eps:
        adr x11, result_bad_eps      // x11 = endereco para
guardar o resultado do caso invalido
        str d0, [x11]               // grava o valor retornado
pela funcao
        adr x11, status_bad_eps      // x11 = endereco para
guardar o status do epsilon invalido
        str w1, [x11]               // grava o status retornado

        adr x9, x_one                // x9 = endereco do valor
x=1.0 para testar max_iter=0
        ldr d0, [x9]                 // d0 = argumento x do seno
        adr x10, eps_small           // x10 = endereco do epsilon
valido
        ldr d1, [x10]               // d1 = epsilon normal
        mov w0, wzr                 // w0 = max_iter=0 para
disparar erro especifico
        bl sin_taylor                // chama a funcao esperando
erro de iteracao

```

```

after_bad_iter:
    adr x11, result_bad_iter           // x11 = endereço para
guardar o resultado do caso max_iter=0
    str d0, [x11]                     // grava o valor retornado
    adr x11, status_bad_iter           // x11 = endereço para
guardar o status do caso max_iter=0
    str w1, [x11]                     // grava o status retornado

    adr x9, status_zero_case           // x9 = endereço do status
do caso x=0
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #0                       // verifica se o caso x=0
foi bem sucedido
    b.ne .fail                         // falha se o status estiver
incorreto

    adr x9, status_one_case            // x9 = endereço do status
do caso x=1
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #0                       // verifica se o caso x=1
foi bem sucedido
    b.ne .fail                         // falha se o status estiver
incorreto

    adr x9, status_bad_eps             // x9 = endereço do status
do epsilon invalido
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #1                       // verifica se o código de
erro foi 1
    b.ne .fail                         // falha se o status estiver
incorreto

    adr x9, status_bad_iter            // x9 = endereço do status
do max_iter invalido
    ldr w10, [x9]                     // carrega o status
retornado

```

```

        cmp w10, #2                                // verifica se o código de
erro foi 2
        b.ne .fail                                // falha se o status estiver
incorreto

        adr x9, result_zero                        // x9 = endereço do
resultado para x=0
        ldr d0, [x9]                              // carrega a aproximação
calculada
        fabs d0, d0                               // toma o valor absoluto do
resultado
        adr x10, tol                              // x10 = endereço da
tolerância numérica
        ldr d1, [x10]                             // carrega a tolerância
        fcmp d0, d1                               // verifica se |sin(0)|
ficou abaixo da tolerância
        b.gt .fail                                // falha se o erro numérico
for alto demais

        adr x9, result_one                        // x9 = endereço do
resultado para x=1
        ldr d0, [x9]                              // carrega a aproximação
calculada
        adr x10, expected_one                    // x10 = endereço do valor
esperado para sin(1)
        ldr d1, [x10]                             // carrega o valor esperado
        fsub d2, d0, d1                           // calcula a diferença entre
aproximado e esperado
        fabs d2, d2                               // toma o valor absoluto do
erro
        adr x11, tol                              // x11 = endereço da
tolerância
        ldr d3, [x11]                             // carrega a tolerância
numérica
        fcmp d2, d3                               // verifica se o erro ficou
dentro da tolerância
        b.gt .fail                                // falha se o erro for maior
que o permitido

```

```

    adr x9, result_bad_eps           // x9 = endereco do
resultado do caso eps invalido
    ldr d0, [x9]                     // carrega o valor retornado
pela funcao
    fabs d0, d0                       // toma o valor absoluto do
retorno
    adr x10, tol                     // x10 = endereco da
tolerancia
    ldr d1, [x10]                    // carrega a tolerancia
    fcmp d0, d1                      // verifica se o retorno
ficou efetivamente em zero
    b.gt .fail                       // falha se o retorno
diferir do esperado

    adr x9, result_bad_iter          // x9 = endereco do
resultado do caso max_iter invalido
    ldr d0, [x9]                     // carrega o valor retornado
pela funcao
    fabs d0, d0                       // toma o valor absoluto do
retorno
    adr x10, tol                     // x10 = endereco da
tolerancia
    ldr d1, [x10]                    // carrega a tolerancia
    fcmp d0, d1                      // verifica se o retorno
ficou efetivamente em zero
    b.gt .fail                       // falha se o retorno
diferir do esperado

    mov w0, wzr                      // w0 = exit code 0
indicando sucesso total
    b .exit                          // encerra o programa com
sucesso

.fail:
    mov w0, #1                       // w0 = exit code 1
indicando falha na validacao

.exit:
    mov x8, #93                      // x8 = syscall exit do
Linux AArch64

```

```

        svc #0                                // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global sin_taylor
.type sin_taylor, %function

sin_taylor:
    fmov d2, xzr                                // d2 = 0.0 para comparacoes
iniciais
    fcmp d1, d2                                // verifica se eps e maior
que zero
    b.le .bad_eps                               // sai com erro se eps <= 0

    cbz w0, .bad_iter                           // sai com erro se max_iter
for zero

    fmov d2, d0                                // d2 = soma acumulada
inicial = primeiro termo
    fmov d3, d0                                // d3 = termo corrente
inicial = x
    fmul d4, d0, d0                            // d4 = x^2 para reutilizar
na recorrência
    mov w2, wzr                                // w2 = contador de termos
adicionais calculados

    fabs d5, d3                                // d5 = |termo atual|
    fcmp d5, d1                                // compara |termo| com eps
    b.lt .done                                 // encerra cedo se o
primeiro termo ja for suficiente

.loop:
    add w2, w2, #1                             // avanca para o proximo
indice k da serie
    cmp w2, w0                                 // verifica se atingiu o
limite de iteracoes

```

```

        b.ge .done                // encerra se nao puder
calcular mais termos

        lsl w5, w2, #1            // w5 = 2k
        add w6, w5, #1            // w6 = 2k + 1
        scvtf d5, w5              // d5 = 2k em ponto
flutuante
        scvtf d6, w6              // d6 = 2k + 1 em ponto
flutuante

        fmul d3, d3, d4           // multiplica o termo atual
por x^2
        fneg d3, d3               // alterna o sinal do termo
        fdiv d3, d3, d5           // divide por 2k
        fdiv d3, d3, d6           // divide por 2k+1,
completando a recorrência
        fadd d2, d2, d3           // soma o novo termo ao
acumulador

        fabs d7, d3               // d7 = |novo termo|
        fcmp d7, d1               // compara |termo| com eps
        b.ge .loop                // continua enquanto o termo
ainda for significativo

.done:
        fmov d0, d2               // retorna a soma acumulada
em d0
        mov w1, wzr               // retorna status 0 em w1
        ret                       // volta ao chamador

.bad_eps:
        fmov d0, xzr              // retorna 0.0 em caso de
epsilon invalido
        mov w1, #1                // retorna status 1 para eps
invalido
        ret                       // volta ao chamador

.bad_iter:
        fmov d0, xzr              // retorna 0.0 em caso de
max_iter invalido

```

```
    mov w1, #2                                // retorna status 2 para
max_iter igual a zero
    ret                                        // volta ao chamador

.section .note.GNU-stack,"",@progbits
```


3. 0

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q3$ qemu-aarch64 -g 2401 ./q3 &
[1] 9035
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q3$ gdb-multiarch -q ./q3
Reading symbols from ./q3...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:31
31      adr x0, vec_a_ok                      // x0 = endereco do vetor A do caso simples
(gdb) break after_overflow_case
Breakpoint 1 at 0x400174: file main.S, line 46.
(gdb) continue
Continuing.

Breakpoint 1, after_overflow_case () at main.S:46
46      adr x9, acc_overflow                  // x9 = endereco para armazenar o acumulador antes do overflow
(gdb) print/u $w1
$1 = 3
(gdb) print/d $x0
$2 = 9223372028264841218
(gdb) continue
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) quit
[1]+  Done                  qemu-aarch64 -g 2401 ./q3
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q3$
```

main.s

```
.data
vec_a_ok:
    .word 1, -2, 3
vec_b_ok:
    .word 4, 5, 6
vec_big_a:
    .word 2147483647, 2147483647, 2147483647
vec_big_b:
    .word 2147483647, 2147483647, 2147483647
.balign 8
acc_ok:
    .quad 0
acc_overflow:
    .quad 0
expected_overflow_acc:
    .quad 9223372028264841218
.balign 4
status_ok:
    .word 0
status_overflow:
    .word 0
status_zero:
    .word 0
status_null:
    .word 0

.text
```

```

.global _start

_start:
    adr x0, vec_a_ok           // x0 = endereço do vetor A
do caso simples
    adr x1, vec_b_ok           // x1 = endereço do vetor B
do caso simples
    mov x2, #3                 // x2 = número de elementos
dos vetores
    bl dot_i32_checked          // calcula o produto escalar
do caso simples
after_ok_case:
    adr x9, acc_ok              // x9 = endereço para
armazenar o acumulador do caso simples
    str x0, [x9]                // grava a soma retornada em
x0
    adr x9, status_ok           // x9 = endereço para
armazenar o status do caso simples
    str w1, [x9]                // grava o status retornado
em w1

    adr x0, vec_big_a           // x0 = endereço do vetor A
do caso de overflow
    adr x1, vec_big_b           // x1 = endereço do vetor B
do caso de overflow
    mov x2, #3                 // x2 = número de elementos
do caso de overflow
    bl dot_i32_checked          // calcula o produto escalar
esperando early-exit
after_overflow_case:
    adr x9, acc_overflow        // x9 = endereço para
armazenar o acumulador antes do overflow
    str x0, [x9]                // grava a soma acumulada
retornada
    adr x9, status_overflow     // x9 = endereço para
armazenar o status do overflow
    str w1, [x9]                // grava o status retornado

    adr x0, vec_a_ok           // x0 = vetor válido para o
teste de N=0

```

```

        adr x1, vec_b_ok                // x1 = outro vetor valido
para o teste
        mov x2, #0                     // x2 = zero elementos para
disparar o erro
        bl dot_i32_checked              // chama a funcao esperando
status 1
after_zero_case:
        adr x9, status_zero             // x9 = endereco para
armazenar o status do caso N=0
        str w1, [x9]                   // grava o status retornado

        mov x0, xzr                    // x0 = ponteiro nulo para
testar erro de ponteiro
        adr x1, vec_b_ok                // x1 = vetor valido para
complementar o teste
        mov x2, #1                     // x2 = um elemento para o
teste de ponteiro nulo
        bl dot_i32_checked              // chama a funcao esperando
status 2
after_null_case:
        adr x9, status_null             // x9 = endereco para
armazenar o status do caso nulo
        str w1, [x9]                   // grava o status retornado

        adr x9, acc_ok                  // x9 = endereco do
acumulador do caso simples
        ldr x10, [x9]                   // carrega o valor calculado
        cmp x10, #12                    // verifica se  $1*4 + (-2)*5$ 
+  $3*6 = 12$ 
        b.ne .fail                      // falha se o resultado
estiver incorreto

        adr x9, status_ok               // x9 = endereco do status
do caso simples
        ldr w10, [x9]                   // carrega o status do caso
simples
        cmp w10, #0                     // verifica se o caso
simples terminou com sucesso
        b.ne .fail                      // falha se o status estiver
incorreto

```

```

    adr x9, status_overflow           // x9 = endereço do status
do caso de overflow
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #3                       // verifica se o overflow
foi sinalizado corretamente
    b.ne .fail                        // falha se o status estiver
incorreto

    adr x9, acc_overflow             // x9 = endereço do
acumulador do caso de overflow
    ldr x10, [x9]                     // carrega a soma acumulada
antes da interrupcao
    adr x11, expected_overflow_acc    // x11 = endereço do valor
esperado
    ldr x12, [x11]                     // carrega o valor esperado
    cmp x10, x12                       // compara acumulador real
com acumulador esperado
    b.ne .fail                        // falha se os valores
divergirem

    adr x9, status_zero              // x9 = endereço do status
do caso N=0
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #1                       // verifica se o erro N=0
foi sinalizado
    b.ne .fail                        // falha se o status estiver
incorreto

    adr x9, status_null              // x9 = endereço do status
do caso de ponteiro nulo
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #2                       // verifica se o erro de
ponteiro foi sinalizado
    b.ne .fail                        // falha se o status estiver
incorreto

```

```

        mov w0, wzr                                // w0 = exit code 0 para
sucesso total
        b .exit                                    // encerra o programa com
sucesso

.fail:
        mov w0, #1                                // w0 = exit code 1 para
falha em alguma validacao

.exit:
        mov x8, #93                                // x8 = syscall exit do
Linux AArch64
        svc #0                                     // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global dot_i32_checked
.type dot_i32_checked, %function

dot_i32_checked:
        cbz x0, .null_ptr                          // retorna erro se o vetor A
for nulo
        cbz x1, .null_ptr                          // retorna erro se o vetor B
for nulo
        cbz x2, .zero_len                          // retorna erro se N for
zero

        mov x3, xzr                                // x3 = acumulador de 64
bits inicializado com zero

.loop:
        ldr w4, [x0], #4                            // le a[i] e avanca o
ponteiro do vetor A
        ldr w5, [x1], #4                            // le b[i] e avanca o
ponteiro do vetor B

```

```

    smull x6, w4, w5           // calcula a[i] * b[i] em 64
bits
    adds x7, x3, x6           // soma o produto ao
acumulador atual atualizando flags
    b.vs .overflow            // interrompe imediatamente
se houver overflow assinado
    mov x3, x7                 // confirma a nova soma no
acumulador
    subs x2, x2, #1           // decrementa a quantidade
de elementos restantes
    b.ne .loop                // repete enquanto ainda
houver elementos

    mov x0, x3                 // retorna o acumulador
final em x0
    mov w1, wzr                // retorna status 0 em w1
    ret                       // volta ao chamador

.overflow:
    mov x0, x3                 // retorna a soma acumulada
antes do overflow
    mov w1, #3                 // retorna status 3 para
overflow detectado
    ret                       // volta ao chamador

.zero_len:
    mov x0, xzr                // retorna soma zero quando
N e invalido
    mov w1, #1                 // retorna status 1 para N=0
    ret                       // volta ao chamador

.null_ptr:
    mov x0, xzr                // retorna soma zero quando
ha ponteiro nulo
    mov w1, #2                 // retorna status 2 para
ponteiro nulo
    ret                       // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

4. O

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q4$ qemu-aarch64 -g 2401 ./q4 &
[1] 9280
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q4$ gdb-multiarch -q ./q4
Reading symbols from ./q4...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:23
23      mov x0, #1000                                // x0 = numerador do caso simples
(gdb) break after_div_zero
Breakpoint 1 at 0x40010c: file main.S, line 36.
(gdb) break after_overflow_guard
Breakpoint 2 at 0x40011c: file main.S, line 46.
(gdb) continue
Continuing.

Breakpoint 1, after_div_zero () at main.S:36
36      adr x9, result_zero_div                      // x9 = endereco para armazenar o retorno do divisor zero
(gdb) print/u $w1
$1 = 1
(gdb) print/d $x0
$2 = 9223372036854775807
(gdb) continue
Continuing.

Breakpoint 2, after_overflow_guard () at main.S:46
46      adr x9, result_overflow_guard                // x9 = endereco para armazenar o retorno do caso extremo
(gdb) print/u $w1
$3 = 2
(gdb) print/d $x0
$4 = 9223372036854775807
(gdb) continue
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) quit
[1]+  Done                  qemu-aarch64 -g 2401 ./q4
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q4$
```

main.s

```
.data
.balign 8
result_ok:
    .quad 0
result_zero_div:
    .quad 0
result_overflow_guard:
    .quad 0
sat_expected:
    .quad 9223372036854775807
.balign 4
status_ok:
    .word 0
status_zero_div:
    .word 0
status_overflow_guard:
    .word 0

.text
.global _start
_start:
    mov x0, #1000                                // x0 = numerador do caso
simples
```

```

    mov x1, #10                                // x1 = denominador do caso
simples
    bl safe_div_s64                            // chama a divisao segura
para o caso normal
after_ok_case:
    adr x9, result_ok                          // x9 = endereco para
armazenar o quociente normal
    str x0, [x9]                              // grava o quociente
retornado
    adr x9, status_ok                          // x9 = endereco para
armazenar o status do caso normal
    str w1, [x9]                              // grava o status retornado

    mov x0, #1000                             // x0 = numerador para o
teste de divisor zero
    mov x1, xzr                                // x1 = divisor zero para
disparar a politica de erro
    bl safe_div_s64                            // chama a divisao segura
esperando saturacao
after_div_zero:
    adr x9, result_zero_div                    // x9 = endereco para
armazenar o retorno do divisor zero
    str x0, [x9]                              // grava o valor saturado
retornado
    adr x9, status_zero_div                    // x9 = endereco para
armazenar o status do divisor zero
    str w1, [x9]                              // grava o status retornado

    mov x0, #1                                // prepara a constante base
para montar INT64_MIN
    lsl x0, x0, #63                           // x0 = INT64_MIN
    mov x1, #-1                               // x1 = -1 para testar o
overflow especial
    bl safe_div_s64                            // chama a divisao segura
esperando saturacao
after_overflow_guard:
    adr x9, result_overflow_guard               // x9 = endereco para
armazenar o retorno do caso extremo
    str x0, [x9]                              // grava o valor saturado
retornado

```



```

    adr x9, status_overflow_guard           // x9 = endereço para
armazenar o status do caso extremo
    str w1, [x9]                           // grava o status retornado

    adr x9, result_ok                     // x9 = endereço do
resultado do caso simples
    ldr x10, [x9]                         // carrega o quociente
calculado
    cmp x10, #100                         // verifica se 1000 / 10 =
100
    b.ne .fail                            // falha se o valor estiver
incorreto

    adr x9, status_ok                     // x9 = endereço do status
do caso simples
    ldr w10, [x9]                         // carrega o status
retornado
    cmp w10, #0                           // verifica se o caso normal
retornou sucesso
    b.ne .fail                            // falha se o status estiver
incorreto

    adr x9, sat_expected                   // x9 = endereço do valor
saturado esperado
    ldr x11, [x9]                         // x11 = constante
0x7FFF_FFFF_FFFF_FFFF

    adr x9, result_zero_div               // x9 = endereço do retorno
para divisor zero
    ldr x10, [x9]                         // carrega o valor retornado
    cmp x10, x11                          // compara com o valor
saturado esperado
    b.ne .fail                            // falha se o valor estiver
incorreto

    adr x9, status_zero_div               // x9 = endereço do status
do divisor zero
    ldr w10, [x9]                         // carrega o status
retornado

```

```

        cmp w10, #1                                // verifica se o código de
erro foi 1
        b.ne .fail                                // falha se o status estiver
incorreto

        adr x9, result_overflow_guard              // x9 = endereço do retorno
do caso extremo
        ldr x10, [x9]                             // carrega o valor retornado
        cmp x10, x11                             // compara com o valor
saturado esperado
        b.ne .fail                                // falha se o valor estiver
incorreto

        adr x9, status_overflow_guard              // x9 = endereço do status
do caso extremo
        ldr w10, [x9]                             // carrega o status
retornado
        cmp w10, #2                                // verifica se o código de
erro foi 2
        b.ne .fail                                // falha se o status estiver
incorreto

        mov w0, wzr                                // w0 = exit code 0 para
sucesso total
        b .exit                                    // encerra o programa com
sucesso

.fail:
        mov w0, #1                                // w0 = exit code 1 para
falha em alguma validacao

.exit:
        mov x8, #93                                // x8 = syscall exit do
Linux AArch64
        svc #0                                     // encerra o processo com o
código em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```
.text
.global safe_div_s64
.type safe_div_s64, %function

safe_div_s64:
    cbz x1, .div_zero           // trata imediatamente o
    caso de divisor zero

    mov x2, #1                  // prepara a base para
    construir INT64_MIN
    lsl x2, x2, #63             // x2 = INT64_MIN
    cmp x0, x2                  // verifica se o numerador e
    INT64_MIN
    b.ne .do_div                // segue para divisao normal
    se nao for o caso especial
    cmp x1, #-1                 // verifica se o denominador
    e -1
    b.ne .do_div                // segue para divisao normal
    se nao for o caso especial

    mov x0, #-1                 // monta um valor com todos
    os bits em 1
    lsr x0, x0, #1               // x0 =
    0x7FFF_FFFF_FFFF_FFFF apos shift logico
    mov w1, #2                   // retorna status 2 para
    overflow evitado
    ret                           // volta ao chamador com
    saturacao positiva

.do_div:
    sdiv x0, x0, x1              // faz a divisao assinada no
    caso normal
    mov w1, wzr                  // retorna status 0 para
    sucesso
    ret                           // volta ao chamador

.div_zero:
    mov x0, #-1                 // monta um valor com todos
    os bits em 1
```

```

    lsr x0, x0, #1                // x0 =
0x7FFF_FFFF_FFFF_FFFF apos shift logico
    mov w1, #1                    // retorna status 1 para
divisor zero
    ret                           // volta ao chamador com
saturacao positiva

.section .note.GNU-stack,"",@progbits

```

5. O

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q5$ qemu-aarch64 -g 2401 ./q5 &
[1] 9979
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q5$ gdb-multiarch -q ./q5
Reading symbols from ./q5...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:23
23      mov w0, #0                      // w0 = modo km/h para m/s
(gdb) break after_mode0
Breakpoint 1 at 0x400150: file main.S, line 27.
(gdb) break after_model1
Breakpoint 2 at 0x40016c: file main.S, line 36.
(gdb) break after_mode2
Breakpoint 3 at 0x400188: file main.S, line 45.
(gdb) break after_invalid_mode
Breakpoint 4 at 0x4001a4: file main.S, line 54.
(gdb) continue
Continuing.

Breakpoint 1, after_mode0 () at main.S:27
27      adr x9, result_mode0           // x9 = endereco para salvar o resultado do modo 0
(gdb) print/u $w0
$1 = 655360
(gdb) print/u $w2
$2 = 0
(gdb) continue
Continuing.

Breakpoint 2, after_model1 () at main.S:36
36      adr x9, result_model1         // x9 = endereco para salvar o resultado do modo 1
(gdb) print/u $w0
$3 = 2359296
(gdb) print/u $w2
$4 = 0
(gdb) continue
Continuing.

Breakpoint 3, after_mode2 () at main.S:45
45      adr x9, result_mode2         // x9 = endereco para salvar o resultado do modo 2
(gdb) print/u $w0
$5 = 121372672
(gdb) print/u $w2
$6 = 0
(gdb) continue
Continuing.

Breakpoint 4, after_invalid_mode () at main.S:54
54      adr x9, result_invalid       // x9 = endereco para salvar o resultado do modo invalido
(gdb) print/u $w0
$7 = 0
(gdb) print/u $w2
$8 = 1
(gdb) continue
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) quit
[1]+  Done                  qemu-aarch64 -g 2401 ./q5
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q5$ |
```

main.s

```
.data
result_mode0:
    .word 0
result_model1:
    .word 0
result_mode2:
    .word 0
result_invalid:
    .word 0
status_mode0:
    .word 0
status_model1:
    .word 0
status_mode2:
```

```

        .word 0
status_invalid:
        .word 0

.text
.global _start

_start:
    mov w0, #0                // w0 = modo km/h para m/s
    mov w1, #2359296          // w1 = 36.0 em Q16.16
    bl convert_speed_q16      // executa a conversao do
modo 0
after_mode0:
    adr x9, result_mode0      // x9 = endereco para salvar
o resultado do modo 0
    str w0, [x9]              // grava o valor convertido
retornado
    adr x9, status_mode0      // x9 = endereco para salvar
o status do modo 0
    str w2, [x9]              // grava o status retornado

    mov w0, #1                // w0 = modo m/s para km/h
    mov w1, #655360           // w1 = 10.0 em Q16.16
    bl convert_speed_q16      // executa a conversao do
modo 1
after_model:
    adr x9, result_model      // x9 = endereco para salvar
o resultado do modo 1
    str w0, [x9]              // grava o valor convertido
retornado
    adr x9, status_model      // x9 = endereco para salvar
o status do modo 1
    str w2, [x9]              // grava o status retornado

    mov w0, #2                // w0 = modo knots para m/s
    mov w1, #235929600        // w1 = valor de teste em
Q16.16
    bl convert_speed_q16      // executa a conversao do
modo 2
after_mode2:

```

```

    adr x9, result_mode2           // x9 = endereço para salvar
o resultado do modo 2
    str w0, [x9]                   // grava o valor convertido
retornado
    adr x9, status_mode2           // x9 = endereço para salvar
o status do modo 2
    str w2, [x9]                   // grava o status retornado

    mov w0, #99                    // w0 = modo invalido para
testar o caso default
    mov w1, #655360                // w1 = valor de entrada
arbitrario em Q16.16
    bl convert_speed_q16           // chama a rotina esperando
erro de modo
after_invalid_mode:
    adr x9, result_invalid         // x9 = endereço para salvar
o resultado do modo invalido
    str w0, [x9]                   // grava o valor retornado
    adr x9, status_invalid         // x9 = endereço para salvar
o status do modo invalido
    str w2, [x9]                   // grava o status retornado

    adr x9, result_mode0           // x9 = endereço do
resultado do modo 0
    ldr w10, [x9]                  // carrega o valor
convertido
    mov w11, #655360               // w11 = 10.0 em Q16.16,
valor esperado
    cmp w10, w11                   // compara resultado
calculado com o esperado
    b.ne .fail                     // falha se o valor estiver
incorreto

    adr x9, status_mode0           // x9 = endereço do status
do modo 0
    ldr w10, [x9]                  // carrega o status
retornado
    cbnz w10, .fail                // falha se o status nao for
zero

```

```

    adr x9, result_model1           // x9 = endereço do
resultado do modo 1
    ldr w10, [x9]                   // carrega o valor
convertido
    mov w11, #2359296               // w11 = 36.0 em Q16.16,
valor esperado
    cmp w10, w11                    // compara resultado
calculado com o esperado
    b.ne .fail                       // falha se o valor estiver
incorreto

    adr x9, status_model1           // x9 = endereço do status
do modo 1
    ldr w10, [x9]                   // carrega o status
retornado
    cbnz w10, .fail                 // falha se o status não for
zero

    adr x9, result_mode2           // x9 = endereço do
resultado do modo 2
    ldr w10, [x9]                   // carrega o valor
convertido
    mov w11, #121372672            // w11 = valor esperado para
a conversão de knots
    cmp w10, w11                    // compara resultado
calculado com o esperado
    b.ne .fail                       // falha se o valor estiver
incorreto

    adr x9, status_mode2           // x9 = endereço do status
do modo 2
    ldr w10, [x9]                   // carrega o status
retornado
    cbnz w10, .fail                 // falha se o status não for
zero

    adr x9, result_invalid         // x9 = endereço do
resultado do modo inválido
    ldr w10, [x9]                   // carrega o valor retornado

```



```

        cbnz w10, .fail           // falha se o retorno nao
for zero

        adr x9, status_invalid   // x9 = endereco do status
do modo invalido
        ldr w10, [x9]           // carrega o status
retornado
        cmp w10, #1             // verifica se o codigo de
erro foi 1
        b.ne .fail             // falha se o status estiver
incorreto

        mov w0, wzr             // w0 = exit code 0 para
sucesso total
        b .exit                // encerra o programa com
sucesso

.fail:
        mov w0, #1             // w0 = exit code 1 para
falha em alguma validacao

.exit:
        mov x8, #93            // x8 = syscall exit do
Linux AArch64
        svc #0                 // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global convert_speed_q16
.type convert_speed_q16, %function

convert_speed_q16:
        cmp w0, #0             // verifica se o modo
solicitado e km/h para m/s
        b.eq .kmh_to_ms        // desvia para a conversao
do modo 0

```

```

    cmp w0, #1                                // verifica se o modo
solicitado e m/s para km/h
    b.eq .ms_to_kmh                           // desvia para a conversao
do modo 1
    cmp w0, #2                                // verifica se o modo
solicitado e knots para m/s
    b.eq .knots_to_ms                         // desvia para a conversao
do modo 2

    mov w0, wzr                               // retorna valor zero para
modo invalido
    mov w2, #1                               // retorna status 1
indicando modo invalido
    ret                                       // volta ao chamador

.kmh_to_ms:
    uxtw x3, w1                              // expande o valor Q16.16
para 64 bits sem sinal
    mov x4, #1000                            // x4 = numerador da
conversao km/h -> m/s
    mul x5, x3, x4                           // multiplica o valor por
1000
    mov x6, #3600                            // x6 = denominador da
conversao
    udiv x5, x5, x6                           // divide por 3600 mantendo
a escala Q16.16
    mov w0, w5                               // retorna o valor
convertido em w0
    mov w2, wzr                               // retorna status 0 para
sucesso
    ret                                       // volta ao chamador

.ms_to_kmh:
    uxtw x3, w1                              // expande o valor Q16.16
para 64 bits sem sinal
    mov x4, #3600                            // x4 = numerador da
conversao m/s -> km/h
    mul x5, x3, x4                           // multiplica o valor por
3600

```

```

        mov x6, #1000                // x6 = denominador da
conversao
        udiv x5, x5, x6              // divide por 1000 mantendo
a escala Q16.16
        mov w0, w5                  // retorna o valor
convertido em w0
        mov w2, wzr                 // retorna status 0 para
sucesso
        ret                         // volta ao chamador

.knots_to_ms:
        uxtw x3, w1                 // expande o valor Q16.16
para 64 bits sem sinal
        mov x4, #1852               // x4 = numerador da
conversao knots -> m/s
        mul x5, x3, x4              // multiplica o valor por
1852
        mov x6, #3600               // x6 = denominador da
conversao
        udiv x5, x5, x6              // divide por 3600 mantendo
a escala Q16.16
        mov w0, w5                  // retorna o valor
convertido em w0
        mov w2, wzr                 // retorna status 0 para
sucesso
        ret                         // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

6. O

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q6$ qemu-aarch64 -g 2401 ./q6 &
[1] 10470
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q6$ gdb-multiarch -q ./q6
Reading symbols from ./q6...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:20
20      adr x0, test_string                // x0 = endereco da string que sera convertida
(gdb) break after_string_case
Breakpoint 1 at 0x40014c: file main.S, line 23.
(gdb) continue
Continuing.

Breakpoint 1, after_string_case () at main.S:23
23      adr x9, count_result              // x9 = endereco para salvar a quantidade convertida
(gdb) print/u $w0
$1 = 3
(gdb) print/u $w1
$2 = 0
(gdb) x/s &test_string
0x41622c: "hello!"
(gdb) c
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) q
[1]+  Done                  qemu-aarch64 -g 2401 ./q6
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q6$ |
```

main.s

```
.data
test_string:
    .ascii "HeLLo!\0"
expected_string:
    .ascii "hello!\0"
.balign 4
count_result:
    .word 0
status_result:
    .word 0
count_null:
    .word 0
status_null:
    .word 0

.text
.global _start

_start:
    adr x0, test_string                // x0 = endereco da string
que sera convertida
    bl to_lower_ascii_inplace          // chama a rotina de
lowercase in-place
after_string_case:
    adr x9, count_result              // x9 = endereco para salvar
a quantidade convertida
```

```

    str w0, [x9] // grava o contador
retornado pela funcao
    adr x9, status_result // x9 = endereco para salvar
o status do caso principal
    str w1, [x9] // grava o status retornado

    mov x0, xzr // x0 = ponteiro nulo para
testar erro de entrada
    bl to_lower_ascii_inplace // chama a rotina esperando
status 1
after_null_case:
    adr x9, count_null // x9 = endereco para salvar
o contador do caso nulo
    str w0, [x9] // grava o contador
retornado
    adr x9, status_null // x9 = endereco para salvar
o status do caso nulo
    str w1, [x9] // grava o status retornado

    adr x10, test_string // x10 = endereco da string
modificada
    adr x11, expected_string // x11 = endereco da string
esperada

.compare_loop:
    ldrb w12, [x10], #1 // le um byte da string
produzida e avanca o ponteiro
    ldrb w13, [x11], #1 // le o byte correspondente
da string esperada
    cmp w12, w13 // compara os bytes atual e
esperado
    b.ne .fail // falha se houver qualquer
divergencia
    cbz w12, .check_status // encerra a comparacao ao
encontrar o terminador nulo
    b .compare_loop // continua comparando os
proximos bytes

.check_status:

```

```

    adr x9, count_result           // x9 = endereco do contador
do caso principal
    ldr w10, [x9]                 // carrega a quantidade de
conversoes realizadas
    cmp w10, #3                   // verifica se exatamente 3
letras foram convertidas
    b.ne .fail                    // falha se o contador
estiver incorreto

    adr x9, status_result         // x9 = endereco do status
do caso principal
    ldr w10, [x9]                 // carrega o status
retornado
    cbnz w10, .fail               // falha se o status nao for
zero

    adr x9, count_null            // x9 = endereco do contador
do caso nulo
    ldr w10, [x9]                 // carrega o contador
retornado
    cbnz w10, .fail               // falha se o contador do
erro nao for zero

    adr x9, status_null           // x9 = endereco do status
do caso nulo
    ldr w10, [x9]                 // carrega o status
retornado
    cmp w10, #1                   // verifica se o codigo de
erro foi 1
    b.ne .fail                    // falha se o status estiver
incorreto

    mov w0, wzr                   // w0 = exit code 0 para
sucesso total
    b .exit                       // encerra o programa com
sucesso

.fail:
    mov w0, #1                    // w0 = exit code 1 para
falha em alguma validacao

```

```

.exit:
    mov x8, #93                // x8 = syscall exit do
Linux AArch64
    svc #0                    // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global to_lower_ascii_inplace
.type to_lower_ascii_inplace, %function

to_lower_ascii_inplace:
    cbz x0, .null_ptr          // retorna erro se o
ponteiro da string for nulo

    mov w2, wzr                // w2 = contador de
caracteres convertidos

.loop:
    ldrb w3, [x0]              // le o byte atual da string
    cbz w3, .done              // encerra ao encontrar o
terminador nulo

    cmp w3, #'A'               // verifica se o caractere e
menor que 'A'
    b.lt .next                 // pula a conversao se nao
for letra maiuscula
    cmp w3, #'Z'               // verifica se o caractere e
maior que 'Z'
    b.gt .next                 // pula a conversao se nao
for letra maiuscula

    add w3, w3, #32             // soma 32 para converter
ASCII maiusculo em minusculo
    strb w3, [x0]              // grava o caractere
convertido de volta na string

```

```

        add w2, w2, #1                // incrementa o contador de
conversoes

.next:
        add x0, x0, #1                // avanca para o proximo
byte da string
        b .loop                      // repete o processamento do
proximo caractere

.done:
        mov w0, w2                   // retorna em w0 a
quantidade de caracteres convertidos
        mov w1, wzr                  // retorna status 0 para
sucesso
        ret                          // volta ao chamador

.null_ptr:
        mov w0, wzr                  // retorna contador zero em
caso de ponteiro nulo
        mov w1, #1                   // retorna status 1 para
ponteiro nulo
        ret                          // volta ao chamador

.section .note.GNU-stack,"",@progbits

```


7. 0

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q7$ qemu-aarch64 -g 2401 ./q7 &
[1] 10725
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q7$ gdb-multiarch -q ./q7
Reading symbols from ./q7...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:29
29      adr x9, a_four                // x9 = endereco do valor 4.0
(gdb) break after_positive_case
Breakpoint 1 at 0x400150: file main.S, line 34.
(gdb) break after_negative_case
Breakpoint 2 at 0x400174: file main.S, line 44.
(gdb) c
Continuing.

Breakpoint 1, after_positive_case () at main.S:34
34      adr x10, result_four          // x10 = endereco para salvar o resultado do caso positivo
(gdb) print/u $w1
$1 = 0
(gdb) print (double)$d0
$2 = 2
(gdb) c
Continuing.

Breakpoint 2, after_negative_case () at main.S:44
44      adr x10, result_neg           // x10 = endereco para salvar o resultado do caso negativo
(gdb) print/u $w1
$3 = 1
(gdb) print (double)$d0
$4 = 0
(gdb) c
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) q
[1]+  Done                  qemu-aarch64 -g 2401 ./q7
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q7$
```

main.s

```
.data
.balign 8
a_four:
    .double 4.0
a_neg:
    .double -1.0
expected_two:
    .double 2.0
tol:
    .double 0.000001
result_four:
    .double 0.0
result_neg:
    .double 0.0
result_zero_iter:
    .double 0.0
.balign 4
status_four:
    .word 0
status_neg:
    .word 0
status_zero_iter:
    .word 0
```

```

.text
.global _start

_start:
    adr x9, a_four           // x9 = endereco do valor
4.0
    ldr d0, [x9]             // d0 = valor de entrada
para a raiz quadrada
    mov w0, #8               // w0 = numero de iteracoes
do metodo
    bl sqrt_nr               // calcula sqrt(4.0) por
Newton-Raphson
after_positive_case:
    adr x10, result_four     // x10 = endereco para
salvar o resultado do caso positivo
    str d0, [x10]            // grava a aproximacao
retornada
    adr x10, status_four     // x10 = endereco para
salvar o status do caso positivo
    str w1, [x10]            // grava o status retornado

    adr x9, a_neg            // x9 = endereco do valor
negativo de teste
    ldr d0, [x9]             // d0 = valor de entrada
invalido
    mov w0, #8               // w0 = iteracoes
arbitrarias para o teste
    bl sqrt_nr               // chama a funcao esperando
erro por entrada negativa
after_negative_case:
    adr x10, result_neg      // x10 = endereco para
salvar o resultado do caso negativo
    str d0, [x10]            // grava o valor retornado
    adr x10, status_neg      // x10 = endereco para
salvar o status do caso negativo
    str w1, [x10]            // grava o status retornado

    adr x9, a_four           // x9 = endereco do valor
4.0 para o teste max_iter=0

```

```

    ldr d0, [x9] // d0 = entrada valida para
esse teste
    mov w0, wzr // w0 = max_iter=0 para
disparar erro especifico
    bl sqrt_nr // chama a funcao esperando
erro de iteracao
after_zero_iter_case:
    adr x10, result_zero_iter // x10 = endereco para
salvar o resultado do caso max_iter=0
    str d0, [x10] // grava o valor retornado
    adr x10, status_zero_iter // x10 = endereco para
salvar o status do caso max_iter=0
    str w1, [x10] // grava o status retornado

    adr x9, status_four // x9 = endereco do status
do caso positivo
    ldr w10, [x9] // carrega o status
retornado
    cbnz w10, .fail // falha se o status nao for
zero

    adr x9, result_four // x9 = endereco do
resultado do caso positivo
    ldr d0, [x9] // carrega a aproximacao
calculada
    adr x10, expected_two // x10 = endereco do valor
esperado 2.0
    ldr d1, [x10] // carrega o valor esperado
    fsub d2, d0, d1 // calcula o erro numerico
da aproximacao
    fabs d2, d2 // toma o valor absoluto do
erro
    adr x11, tol // x11 = endereco da
tolerancia
    ldr d3, [x11] // carrega a tolerancia
numerica
    fcmp d2, d3 // compara o erro com a
tolerancia
    b.gt .fail // falha se o erro for maior
que o permitido

```

```

    adr x9, status_neg                // x9 = endereco do status
do caso negativo
    ldr w10, [x9]                    // carrega o status
retornado
    cmp w10, #1                      // verifica se o codigo de
erro foi 1
    b.ne .fail                       // falha se o status estiver
incorreto

    adr x9, result_neg               // x9 = endereco do
resultado do caso negativo
    ldr d0, [x9]                    // carrega o retorno da
funcao
    fabs d0, d0                     // toma o valor absoluto do
retorno
    adr x10, tol                    // x10 = endereco da
tolerancia
    ldr d1, [x10]                   // carrega a tolerancia
    fcmp d0, d1                     // verifica se o retorno
ficou em zero
    b.gt .fail                       // falha se o retorno
estiver fora do esperado

    adr x9, status_zero_iter        // x9 = endereco do status
do caso max_iter=0
    ldr w10, [x9]                    // carrega o status
retornado
    cmp w10, #2                      // verifica se o codigo de
erro foi 2
    b.ne .fail                       // falha se o status estiver
incorreto

    adr x9, result_zero_iter        // x9 = endereco do
resultado do caso max_iter=0
    ldr d0, [x9]                    // carrega o retorno da
funcao
    fabs d0, d0                     // toma o valor absoluto do
retorno

```

```

        adr x10, tol                // x10 = endereco da
tolerancia
        ldr d1, [x10]              // carrega a tolerancia
        fcmp d0, d1                // verifica se o retorno
ficou em zero
        b.gt .fail                 // falha se o retorno
estiver fora do esperado

        mov w0, wzr                // w0 = exit code 0 para
sucesso total
        b .exit                    // encerra o programa com
sucesso

.fail:
        mov w0, #1                 // w0 = exit code 1 para
falha em alguma validacao

.exit:
        mov x8, #93                // x8 = syscall exit do
Linux AArch64
        svc #0                     // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global sqrt_nr
.type sqrt_nr, %function

sqrt_nr:
        fmov d2, xzr               // d2 = 0.0 para a validacao
inicial
        fcmp d0, d2                // verifica se a entrada e
negativa
        b.lt .bad_input            // sai com erro se a < 0

        cbz w0, .bad_iter           // sai com erro se max_iter
for zero

```

```

    fmov d4, d0                                // d4 = copia de 'a' para
usar na recorrência
    fmov d3, #1.0                              // d3 = constante 1.0 para o
chute mínimo
    fcmp d0, d3                                // verifica se a >= 1.0
    b.ge .use_a_as_guess                       // usa a própria entrada
como chute se ela for >= 1

    fmov d2, d3                                // d2 = chute inicial 1.0
para evitar divisão por zero
    b .loop_setup                              // vai para a preparação do
laco

.use_a_as_guess:
    fmov d2, d0                                // d2 = chute inicial igual
ao próprio valor 'a'

.loop_setup:
    mov w2, w0                                // w2 = contador de
iterações restantes
    fmov d5, #0.5                              // d5 = constante 0.5 usada
na média da iteração

.loop:
    fdiv d6, d4, d2                            // d6 = a / x_n
    fadd d2, d2, d6                            // soma x_n + a/x_n
    fmul d2, d2, d5                            // calcula x_(n+1) = 0.5 *
(x_n + a/x_n)
    subs w2, w2, #1                            // decrementa o número de
iterações restantes
    b.ne .loop                                 // continua até completar
max_iter passos

    fmov d0, d2                                // retorna a aproximação
final em d0
    mov w1, wzr                                // retorna status 0 para
sucesso
    ret                                         // volta ao chamador

```

```

.bad_input:
    fmov d0, xzr                // retorna 0.0 em caso de
entrada invalida
    mov w1, #1                  // retorna status 1 para a <
0
    ret                         // volta ao chamador

.bad_iter:
    fmov d0, xzr                // retorna 0.0 em caso de
max_iter invalido
    mov w1, #2                  // retorna status 2 para
max_iter igual a zero
    ret                         // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

8. 0

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q8$ qemu-aarch64 -g 2401 ./q8 &
[1] 10979
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q8$ gdb-multiarch -q ./q8
Reading symbols from ./q8...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:35
35      adr x0, signal                      // x0 = endereco do vetor de amostras do caso principal
(gdb) break after_primary_case
Breakpoint 1 at 0x400130: file main.S, line 41.
(gdb) c
Undefined command: "C". Try "help".
(gdb) c
Continuing.

Breakpoint 1, after_primary_case () at main.S:41
41      adr x10, result_ok                  // x10 = endereco para salvar a integral calculada
(gdb) print/u $w0
$1 = 0
(gdb) print (double)$d0
$2 = 2
(gdb) c
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) q
[1]+  Done                  qemu-aarch64 -g 2401 ./q8
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q8$ |
```

main.s

```
.data
.balign 8
signal:
    .double 0.0, 1.0, 2.0
dt_one:
    .double 1.0
dt_zero:
    .double 0.0
expected_integral:
    .double 2.0
tol:
    .double 0.000001
result_ok:
    .double 0.0
result_bad_n:
    .double 0.0
result_bad_dt:
    .double 0.0
result_null:
    .double 0.0
.balign 4
status_ok:
    .word 0
status_bad_n:
    .word 0
status_bad_dt:
    .word 0
```



```

status_null:
    .word 0

.text
.global _start

_start:
    adr x0, signal                // x0 = endereco do vetor de
amostras do caso principal
    mov x1, #3                    // x1 = quantidade de
amostras do caso principal
    adr x9, dt_one                // x9 = endereco do dt
valido
    ldr d0, [x9]                  // d0 = passo de integracao
    bl trapz_energy_f64           // calcula a integral pelo
metodo do trapezio
after_primary_case:
    adr x10, result_ok            // x10 = endereco para
salvar a integral calculada
    str d0, [x10]                // grava o resultado
retornado
    adr x10, status_ok            // x10 = endereco para
salvar o status do caso principal
    str w0, [x10]                // grava o status retornado

    adr x0, signal                // x0 = vetor valido para o
teste N<2
    mov x1, #1                    // x1 = numero insuficiente
de amostras
    adr x9, dt_one                // x9 = endereco do dt
valido
    ldr d0, [x9]                  // d0 = passo de integracao
    bl trapz_energy_f64           // chama a funcao esperando
status 2
after_bad_n_case:
    adr x10, result_bad_n         // x10 = endereco para
salvar o resultado do caso N invalido
    str d0, [x10]                // grava o retorno da funcao
    adr x10, status_bad_n         // x10 = endereco para
salvar o status do caso N invalido

```

```

    str w0, [x10]                                // grava o status retornado

    adr x0, signal                                // x0 = vetor valido para o
teste de dt invalido
    mov x1, #3                                    // x1 = quantidade de
amostras suficiente
    adr x9, dt_zero                               // x9 = endereco do dt zero
    ldr d0, [x9]                                  // d0 = passo invalido
    bl trapz_energy_f64                           // chama a funcao esperando
status 3
after_bad_dt_case:
    adr x10, result_bad_dt                       // x10 = endereco para
salvar o resultado do caso dt invalido
    str d0, [x10]                                // grava o retorno da funcao
    adr x10, status_bad_dt                       // x10 = endereco para
salvar o status do caso dt invalido
    str w0, [x10]                                // grava o status retornado

    mov x0, xzr                                  // x0 = ponteiro nulo para
testar erro de entrada
    mov x1, #3                                    // x1 = numero valido de
amostras
    adr x9, dt_one                               // x9 = endereco do dt
valido
    ldr d0, [x9]                                  // d0 = passo de integracao
    bl trapz_energy_f64                           // chama a funcao esperando
status 1
after_null_case:
    adr x10, result_null                         // x10 = endereco para
salvar o resultado do caso nulo
    str d0, [x10]                                // grava o retorno da funcao
    adr x10, status_null                         // x10 = endereco para
salvar o status do caso nulo
    str w0, [x10]                                // grava o status retornado

    adr x9, status_ok                            // x9 = endereco do status
do caso principal
    ldr w10, [x9]                                // carrega o status
retornado

```

```

    cbnz w10, .fail // falha se o status nao for
zero

    adr x9, result_ok // x9 = endereco da integral
calculada
    ldr d0, [x9] // carrega o valor retornado
    adr x10, expected_integral // x10 = endereco da
integral esperada
    ldr d1, [x10] // carrega o valor esperado
    fsub d2, d0, d1 // calcula o erro numerico
da integral
    fabs d2, d2 // toma o valor absoluto do
erro
    adr x11, tol // x11 = endereco da
tolerancia
    ldr d3, [x11] // carrega a tolerancia
numerica
    fcmp d2, d3 // compara o erro com a
tolerancia
    b.gt .fail // falha se o erro for maior
que o permitido

    adr x9, status_bad_n // x9 = endereco do status
do caso N invalido
    ldr w10, [x9] // carrega o status
retornado
    cmp w10, #2 // verifica se o codigo de
erro foi 2
    b.ne .fail // falha se o status estiver
incorreto

    adr x9, result_bad_n // x9 = endereco do
resultado do caso N invalido
    ldr d0, [x9] // carrega o retorno da
funcao
    fabs d0, d0 // toma o valor absoluto do
retorno
    adr x10, tol // x10 = endereco da
tolerancia
    ldr d1, [x10] // carrega a tolerancia

```

```

        fcmp d0, d1                                // verifica se o retorno
ficou em zero
        b.gt .fail                                // falha se o retorno
estiver fora do esperado

        adr x9, status_bad_dt                     // x9 = endereco do status
do caso dt invalido
        ldr w10, [x9]                             // carrega o status
retornado
        cmp w10, #3                               // verifica se o codigo de
erro foi 3
        b.ne .fail                                // falha se o status estiver
incorreto

        adr x9, result_bad_dt                     // x9 = endereco do
resultado do caso dt invalido
        ldr d0, [x9]                             // carrega o retorno da
funcao
        fabs d0, d0                               // toma o valor absoluto do
retorno
        adr x10, tol                             // x10 = endereco da
tolerancia
        ldr d1, [x10]                             // carrega a tolerancia
        fcmp d0, d1                               // verifica se o retorno
ficou em zero
        b.gt .fail                                // falha se o retorno
estiver fora do esperado

        adr x9, status_null                       // x9 = endereco do status
do caso nulo
        ldr w10, [x9]                             // carrega o status
retornado
        cmp w10, #1                               // verifica se o codigo de
erro foi 1
        b.ne .fail                                // falha se o status estiver
incorreto

        adr x9, result_null                       // x9 = endereco do
resultado do caso nulo

```

```

    ldr d0, [x9]                // carrega o retorno da
funcao                          //
    fabs d0, d0                 // toma o valor absoluto do
retorno                         //
    adr x10, tol                // x10 = endereco da
tolerancia                      //
    ldr d1, [x10]               // carrega a tolerancia
    fcmp d0, d1                 // verifica se o retorno
ficou em zero                   //
    b.gt .fail                  // falha se o retorno
estiver fora do esperado        //

    mov w0, wzr                 // w0 = exit code 0 para
sucesso total                   //
    b .exit                     // encerra o programa com
sucesso                         //

.fail:
    mov w0, #1                  // w0 = exit code 1 para
falha em alguma validacao      //

.exit:
    mov x8, #93                 // x8 = syscall exit do
Linux AArch64                  //
    svc #0                      // encerra o processo com o
codigo em w0                   //

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global trapz_energy_f64
.type trapz_energy_f64, %function

trapz_energy_f64:
    cbz x0, .null_ptr           // retorna erro se o
ponteiro do vetor for nulo    //
    cmp x1, #2                  // verifica se ha ao menos
duas amostras                  //

```

```

    b.lt .bad_n                                // retorna erro se N < 2

    fmov d1, xzr                                // d1 = 0.0 para validar o
passo dt
    fcmp d0, d1                                // compara dt com zero
    b.le .bad_dt                                // retorna erro se dt <= 0

    mov x2, x0                                  // x2 = ponteiro de trabalho
para percorrer o vetor
    sub x3, x1, #1                              // x3 = numero de trapezios
a acumular
    ldr d2, [x2]                                // d2 = f[i] inicial
    fmov d3, xzr                                // d3 = acumulador da
integral inicializado com zero
    fmov d1, #0.5                              // d1 = constante 0.5 da
regra do trapezio

.loop:
    ldr d4, [x2, #8]                            // d4 = f[i+1] usando o
proximo elemento do vetor
    fadd d5, d2, d4                            // soma f[i] + f[i+1]
    fmul d5, d5, d0                            // multiplica pela largura
dt
    fmul d5, d5, d1                            // aplica o fator 0.5 do
trapezio
    fadd d3, d3, d5                            // acumula a area parcial no
total
    fmov d2, d4                                // atualiza f[i] para a
proxima iteracao
    add x2, x2, #8                              // avanca o ponteiro para o
proximo elemento
    subs x3, x3, #1                            // decrementa a quantidade
de trapezios restantes
    b.ne .loop                                // continua enquanto ainda
houver pares para integrar

    fmov d0, d3                                // retorna a integral
aproximada em d0
    mov w0, wzr                                // retorna status 0 para
sucesso

```

```

    ret                                // volta ao chamador

.null_ptr:
    fmov d0, xzr                      // retorna 0.0 em caso de
ponteiro nulo
    mov w0, #1                        // retorna status 1 para
ponteiro nulo
    ret                                // volta ao chamador

.bad_n:
    fmov d0, xzr                      // retorna 0.0 em caso de N
invalido
    mov w0, #2                        // retorna status 2 para N<2
    ret                                // volta ao chamador

.bad_dt:
    fmov d0, xzr                      // retorna 0.0 em caso de dt
invalido
    mov w0, #3                        // retorna status 3 para
dt<=0
    ret                                // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

9. O

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q9$ qemu-aarch64 -g 2401 ./q9 &
[1] 11495
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q9$ gdb-multiarch -q ./q9
Reading symbols from ./q9...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:29
29      mov w0, #0                                // w0 = mode 0 para calcular apenas A*p
(gdb) break after_model_case
Breakpoint 1 at 0x40017c: file main.S, line 46.
(gdb) c
Continuing.

Breakpoint 1, after_model_case () at main.S:46
46      adr x9, status_model                      // x9 = endereço para salvar o status do modo 1
(gdb) print/u $w0
$1 = 0
(gdb) x/2wd &out_model
0x410328:      196608 262144
(gdb) c
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) q
[1]+  Done                  qemu-aarch64 -g 2401 ./q9
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q9$
```

main.s

```
.data
a_matrix:
    .word 65536, 0, 0, 65536
b_vec:
    .word 65536, 65536
p_vec:
    .word 131072, 196608
out_mode0:
    .word 0, 0
out_model:
    .word 0, 0
out_invalid:
    .word 0, 0
expected_mode0:
    .word 131072, 196608
expected_model:
    .word 196608, 262144
status_mode0:
    .word 0
status_model:
    .word 0
status_invalid:
    .word 0

.text
.global _start
```



```

_start:
    mov w0, #0                // w0 = mode 0 para calcular
apenas A*p
    adr x1, a_matrix          // x1 = endereco da matriz A
    adr x2, b_vec             // x2 = endereco do vetor b
    adr x3, p_vec             // x3 = endereco do ponto de
entrada p
    adr x4, out_mode0         // x4 = endereco do vetor de
saida do modo 0
    bl affine2d_q16           // executa a transformacao
no modo 0
after_mode0_case:
    adr x9, status_mode0     // x9 = endereco para salvar
o status do modo 0
    str w0, [x9]              // grava o status retornado

    mov w0, #1                // w0 = mode 1 para calcular
A*p + b
    adr x1, a_matrix          // x1 = endereco da matriz A
    adr x2, b_vec             // x2 = endereco do vetor b
    adr x3, p_vec             // x3 = endereco do ponto de
entrada p
    adr x4, out_model1        // x4 = endereco do vetor de
saida do modo 1
    bl affine2d_q16           // executa a transformacao
no modo 1
after_model1_case:
    adr x9, status_model1     // x9 = endereco para salvar
o status do modo 1
    str w0, [x9]              // grava o status retornado

    mov w0, #99               // w0 = modo invalido para
testar o caso default
    adr x1, a_matrix          // x1 = endereco da matriz A
    adr x2, b_vec             // x2 = endereco do vetor b
    adr x3, p_vec             // x3 = endereco do ponto de
entrada p
    adr x4, out_invalid       // x4 = endereco do vetor de
saida para o caso invalido

```

```

        bl affine2d_q16                                // chama a funcao esperando
erro de modo
after_invalid_mode:
        adr x9, status_invalid                        // x9 = endereco para salvar
o status do modo invalido
        str w0, [x9]                                  // grava o status retornado

        adr x10, out_mode0                            // x10 = endereco da saida
produzida no modo 0
        adr x11, expected_mode0                      // x11 = endereco da saida
esperada no modo 0
        mov x12, #2                                    // x12 = contador dos 2
elementos do vetor 2D

.check_mode0:
        ldr w13, [x10], #4                            // le o proximo elemento
produzido no modo 0
        ldr w14, [x11], #4                            // le o elemento esperado
correspondente
        cmp w13, w14                                  // compara saida real com
saida esperada
        b.ne .fail                                    // falha se houver
divergencia
        subs x12, x12, #1                              // decrementa o contador de
elementos restantes
        b.ne .check_mode0                            // continua enquanto ainda
houver elementos a validar

        adr x10, out_model1                          // x10 = endereco da saida
produzida no modo 1
        adr x11, expected_model1                    // x11 = endereco da saida
esperada no modo 1
        mov x12, #2                                    // x12 = contador dos 2
elementos do vetor 2D

.check_model1:
        ldr w13, [x10], #4                            // le o proximo elemento
produzido no modo 1
        ldr w14, [x11], #4                            // le o elemento esperado
correspondente

```

```

    cmp w13, w14                                // compara saida real com
saida esperada
    b.ne .fail                                  // falha se houver
divergencia
    subs x12, x12, #1                            // decrementa o contador de
elementos restantes
    b.ne .check_model                            // continua enquanto ainda
houver elementos a validar

    adr x9, status_mode0                        // x9 = endereco do status
do modo 0
    ldr w10, [x9]                                // carrega o status
retornado
    cbnz w10, .fail                              // falha se o status nao for
zero

    adr x9, status_model1                        // x9 = endereco do status
do modo 1
    ldr w10, [x9]                                // carrega o status
retornado
    cbnz w10, .fail                              // falha se o status nao for
zero

    adr x9, status_invalid                       // x9 = endereco do status
do modo invalido
    ldr w10, [x9]                                // carrega o status
retornado
    cmp w10, #2                                // verifica se o codigo de
erro foi 2
    b.ne .fail                                  // falha se o status estiver
incorreto

    mov w0, wzr                                  // w0 = exit code 0 para
sucesso total
    b .exit                                      // encerra o programa com
sucesso

.fail:
    mov w0, #1                                  // w0 = exit code 1 para
falha em alguma validacao

```

```

.exit:
    mov x8, #93                // x8 = syscall exit do
Linux AArch64
    svc #0                    // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global affine2d_q16
.type affine2d_q16, %function

affine2d_q16:
    cbz x1, .null_ptr          // retorna erro se o
ponteiro da matriz A for nulo
    cbz x2, .null_ptr          // retorna erro se o
ponteiro do vetor b for nulo
    cbz x3, .null_ptr          // retorna erro se o
ponteiro do ponto p for nulo
    cbz x4, .null_ptr          // retorna erro se o
ponteiro da saida for nulo

    cmp w0, #0                 // verifica se o modo
solicitado e A*p
    b.eq .mode_ap              // desvia para o modo 0
    cmp w0, #1                 // verifica se o modo
solicitado e A*p+b
    b.eq .mode_apb             // desvia para o modo 1

    mov w0, #2                 // retorna status 2 para
modo invalido
    ret                        // volta ao chamador

.mode_ap:
    ldr w5, [x1]               // carrega a00
    ldr w6, [x1, #4]           // carrega a01
    ldr w7, [x1, #8]           // carrega a10

```

```

    ldr w8, [x1, #12]           // carrega a11
    ldr w9, [x3]                // carrega p.x
    ldr w10, [x3, #4]           // carrega p.y

    smull x11, w5, w9            // calcula a00 * p.x em 64
bits
    smull x12, w6, w10           // calcula a01 * p.y em 64
bits
    add x11, x11, x12           // soma os termos da
primeira linha
    asr x11, x11, #16            // normaliza o resultado de
volta para Q16.16

    smull x13, w7, w9            // calcula a10 * p.x em 64
bits
    smull x14, w8, w10           // calcula a11 * p.y em 64
bits
    add x13, x13, x14           // soma os termos da segunda
linha
    asr x13, x13, #16            // normaliza o resultado de
volta para Q16.16

    str w11, [x4]                // grava out.x na memoria de
saida
    str w13, [x4, #4]            // grava out.y na memoria de
saida
    mov w0, wzr                 // retorna status 0 para
sucesso
    ret                          // volta ao chamador

.mode_apb:
    ldr w5, [x1]                // carrega a00
    ldr w6, [x1, #4]             // carrega a01
    ldr w7, [x1, #8]             // carrega a10
    ldr w8, [x1, #12]            // carrega a11
    ldr w9, [x3]                // carrega p.x
    ldr w10, [x3, #4]            // carrega p.y
    ldr w15, [x2]                // carrega b.x
    ldr w16, [x2, #4]            // carrega b.y

```

```

    smull x11, w5, w9           // calcula a00 * p.x em 64
bits
    smull x12, w6, w10         // calcula a01 * p.y em 64
bits
    add x11, x11, x12          // soma os termos da
primeira linha
    asr x11, x11, #16           // normaliza o resultado de
volta para Q16.16
    sxtw x15, w15               // expande b.x para 64 bits
com sinal
    add x11, x11, x15           // soma a translacao b.x ao
resultado x

    smull x13, w7, w9           // calcula a10 * p.x em 64
bits
    smull x14, w8, w10         // calcula a11 * p.y em 64
bits
    add x13, x13, x14          // soma os termos da segunda
linha
    asr x13, x13, #16           // normaliza o resultado de
volta para Q16.16
    sxtw x16, w16               // expande b.y para 64 bits
com sinal
    add x13, x13, x16           // soma a translacao b.y ao
resultado y

    str w11, [x4]               // grava out.x na memoria de
saida
    str w13, [x4, #4]           // grava out.y na memoria de
saida
    mov w0, wzr                 // retorna status 0 para
sucesso
    ret                          // volta ao chamador

.null_ptr:
    mov w0, #1                  // retorna status 1 para
ponteiro nulo
    ret                          // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

10. O

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q10$ qemu-aarch64 -g 2401 ./q10 &
[1] 11723
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q10$ gdb-multiarch -q ./q10
Reading symbols from ./q10...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:21
21      adr x0, input_vec                // x0 = endereco do vetor de entrada
(gdb) break after_primary_case
Breakpoint 1 at 0x400150: file main.S, line 27.
(gdb) c
Undefined command: "C". Try "help".
(gdb) c
Continuing.
Breakpoint 1, after_primary_case () at main.S:27
27      adr x9, status_ok                // x9 = endereco para salvar o status do caso principal
(gdb) print/u $w0
$1 = 0
(gdb) x/4wd &output_vec
0x4102dc: 0 1 2 3
(gdb) c
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) q
[1]+  Done                  qemu-aarch64 -g 2401 ./q10
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q10$ |
```

main.s

```
.data
input_vec:
    .word 1, 2, 3, 4
output_vec:
    .word 0, 0, 0, 0
expected_vec:
    .word 0, 1, 2, 3
status_ok:
    .word 0
status_bad_w:
    .word 0
status_bad_n:
    .word 0
status_null:
    .word 0

.text
.global _start

_start:
    adr x0, input_vec                // x0 = endereco do vetor de
entrada
    adr x1, output_vec              // x1 = endereco do vetor de
saida
    mov x2, #4                      // x2 = numero de elementos
do vetor
    mov w3, #2                      // w3 = tamanho da janela
```

```

        bl moving_avg_i32                // calcula a media movel do
caso principal
after_primary_case:
        adr x9, status_ok                // x9 = endereco para salvar
o status do caso principal
        str w0, [x9]                    // grava o status retornado

        adr x0, input_vec                // x0 = vetor valido para
testar janela invalida
        adr x1, output_vec              // x1 = saida valida para o
teste
        mov x2, #4                      // x2 = numero de elementos
valido
        mov w3, wzr                     // w3 = janela zero para
disparar erro
        bl moving_avg_i32                // chama a rotina esperando
status 3
after_bad_w_case:
        adr x9, status_bad_w            // x9 = endereco para salvar
o status do W invalido
        str w0, [x9]                    // grava o status retornado

        adr x0, input_vec                // x0 = vetor valido para
testar N=0
        adr x1, output_vec              // x1 = saida valida para o
teste
        mov x2, xzr                     // x2 = N=0 para disparar
erro especifico
        mov w3, #2                      // w3 = janela valida
        bl moving_avg_i32                // chama a rotina esperando
status 2
after_bad_n_case:
        adr x9, status_bad_n            // x9 = endereco para salvar
o status do N invalido
        str w0, [x9]                    // grava o status retornado

        mov x0, xzr                     // x0 = ponteiro nulo para
testar erro de entrada
        adr x1, output_vec              // x1 = saida valida para o
teste

```



```

        mov x2, #4                                // x2 = numero de elementos
valido
        mov w3, #2                                // w3 = janela valida
        bl moving_avg_i32                        // chama a rotina esperando
status 1
after_null_case:
        adr x9, status_null                      // x9 = endereco para salvar
o status do caso nulo
        str w0, [x9]                            // grava o status retornado

        adr x10, output_vec                     // x10 = endereco da saida
produzida
        adr x11, expected_vec                   // x11 = endereco da saida
esperada
        mov x12, #4                             // x12 = contador de
elementos do vetor

.check_loop:
        ldr w13, [x10], #4                      // le um elemento produzido
e avanca o ponteiro
        ldr w14, [x11], #4                      // le o elemento esperado
correspondente
        cmp w13, w14                           // compara os dois valores
        b.ne .fail                             // falha se houver
divergencia
        subs x12, x12, #1                       // decrementa o contador de
elementos restantes
        b.ne .check_loop                       // continua enquanto ainda
houver elementos a validar

        adr x9, status_ok                      // x9 = endereco do status
do caso principal
        ldr w10, [x9]                          // carrega o status
retornado
        cbnz w10, .fail                        // falha se o status nao for
zero

        adr x9, status_bad_w                   // x9 = endereco do status
do W invalido

```

```

    ldr w10, [x9]                // carrega o status
retornado
    cmp w10, #3                  // verifica se o código de
erro foi 3
    b.ne .fail                   // falha se o status estiver
incorreto

    adr x9, status_bad_n        // x9 = endereço do status
do N inválido
    ldr w10, [x9]                // carrega o status
retornado
    cmp w10, #2                  // verifica se o código de
erro foi 2
    b.ne .fail                   // falha se o status estiver
incorreto

    adr x9, status_null         // x9 = endereço do status
do caso nulo
    ldr w10, [x9]                // carrega o status
retornado
    cmp w10, #1                  // verifica se o código de
erro foi 1
    b.ne .fail                   // falha se o status estiver
incorreto

    mov w0, wzr                  // w0 = exit code 0 para
sucesso total
    b .exit                       // encerra o programa com
sucesso

.fail:
    mov w0, #1                   // w0 = exit code 1 para
falha em alguma validação

.exit:
    mov x8, #93                  // x8 = syscall exit do
Linux AArch64
    svc #0                       // encerra o processo com o
código em w0

```

```
.section .note.GNU-stack,"",@progbits
```

lib.s

```
.text
.global moving_avg_i32
.type moving_avg_i32, %function

moving_avg_i32:
    cbz x0, .null_ptr           // retorna erro se o
    // ponteiro de entrada for nulo
    cbz x1, .null_ptr           // retorna erro se o
    // ponteiro de saída for nulo
    cbz x2, .zero_len           // retorna erro se N for
    // zero
    cbz w3, .bad_window         // retorna erro se a janela
    // W for zero

    uxtw x6, w3                 // x6 = tamanho da janela
    // expandido para 64 bits
    cmp x6, x2                  // verifica se W <= N
    b.hi .bad_window            // retorna erro se a janela
    // for maior que o vetor

    mov x4, xzr                 // x4 = índice i
    // inicializado com zero
    mov x5, xzr                 // x5 = acumulador
    // deslizando da janela
    sub w7, w3, #1              // w7 = W - 1 para o teste
    // de borda
    uxtw x7, w7                 // x7 = W - 1 expandido para
    // 64 bits

.loop:
    cmp x4, x2                  // verifica se i já atingiu
    // N
    b.ge .done                 // encerra quando todo o
    // vetor tiver sido processado
```

```

    add x8, x0, x4, lsl #2           // calcula o endereco de
in[i]
    ldr w9, [x8]                     // le o elemento atual do
vetor de entrada
    sxtw x9, w9                      // expande o elemento atual
para 64 bits com sinal
    add x5, x5, x9                   // soma o novo elemento ao
acumulador da janela

    cmp x4, x6                       // verifica se a janela ja
passou do primeiro bloco completo
    b.lt .no_sub                     // nao subtrai nada antes de
i >= W
    sub x10, x4, x6                  // calcula o indice do
elemento que sai da janela
    add x11, x0, x10, lsl #2         // calcula o endereco de
in[i-W]
    ldr w12, [x11]                   // le o elemento que sai da
janela
    sxtw x12, w12                    // expande o elemento
removido para 64 bits com sinal
    sub x5, x5, x12                  // remove esse elemento do
acumulador

.no_sub:
    add x13, x1, x4, lsl #2         // calcula o endereco de
out[i]
    cmp x4, x7                       // verifica se i ainda esta
antes de W-1
    b.lt .store_zero                // grava zero enquanto a
janela ainda nao estiver completa

    sdiv x14, x5, x6                 // calcula a media inteira
da janela atual
    str w14, [x13]                   // grava a media calculada
na saida
    b .next                          // segue para a proxima
iteracao

.store_zero:

```

```

        str w0, [x13]                // grava zero para as
posicoes antes da janela completa

.next:
        add x4, x4, #1                // avanca o indice i
        b .loop                      // repete o processamento do
proximo elemento

.done:
        mov w0, w0                    // retorna status 0 para
sucesso
        ret                          // volta ao chamador

.null_ptr:
        mov w0, #1                    // retorna status 1 para
ponteiro nulo
        ret                          // volta ao chamador

.zero_len:
        mov w0, #2                    // retorna status 2 para N=0
        ret                          // volta ao chamador

.bad_window:
        mov w0, #3                    // retorna status 3 para
janela invalida
        ret                          // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

11.0

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/Faculdade/26.1-sistemasembarcados/drs/dr2/at/q11$ qemu-aarch64 -g 2401 ./q11 &
[1] 11955
renato@NOTE-PESSOAL:~/Faculdade/26.1-sistemasembarcados/drs/dr2/at/q11$ gdb-multiarch -q ./q11
Reading symbols from ./q11...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:27
27      adr x0, coef_ok                      // x0 = endereco dos coeficientes do caso simples
(gdb) break after_simple_case
Breakpoint 1 at 0x400134: file main.S, line 32.
(gdb) break after_saturation_case
Breakpoint 2 at 0x400174: file main.S, line 42.
(gdb) c
Continuing.

Breakpoint 1, after_simple_case () at main.S:32
32      adr x9, result_ok                    // x9 = endereco para salvar o resultado do caso simples
(gdb) print/u $w1
$1 = 0
(gdb) print/d $w0
$2 = 2949120
(gdb) c
Continuing.

Breakpoint 2, after_saturation_case () at main.S:42
42      adr x9, result_sat                  // x9 = endereco para salvar o resultado saturado
(gdb) print/u $w1
$3 = 3
(gdb) print/d $w0
$4 = 2147483647
(gdb) c
Continuing.
[Inferior 1 (process 1) exited normally]
(gdb) q
[1]+  Done                  qemu-aarch64 -g 2401 ./q11
renato@NOTE-PESSOAL:~/Faculdade/26.1-sistemasembarcados/drs/dr2/at/q11$
```

main.s

```
.data
coef_ok:
    .word 131072, 196608, 65536
coef_sat:
    .word 2147483647, 2147483647
result_ok:
    .word 0
result_sat:
    .word 0
result_bad_n:
    .word 0
result_null:
    .word 0
status_ok:
    .word 0
status_sat:
    .word 0
status_bad_n:
    .word 0
status_null:
    .word 0

.text
.global _start
```

```

_start:
    adr x0, coef_ok                // x0 = endereco dos
coeficientes do caso simples
    mov x1, #3                    // x1 = numero de
coeficientes do polinomio simples
    mov w2, #262144               // w2 = x = 4.0 em Q16.16
    bl poly_horner_q16_sat        // avalia o polinomio pelo
metodo de Horner
after_simple_case:
    adr x9, result_ok            // x9 = endereco para salvar
o resultado do caso simples
    str w0, [x9]                 // grava o valor do
polinomio retornado
    adr x9, status_ok            // x9 = endereco para salvar
o status do caso simples
    str w1, [x9]                 // grava o status retornado

    adr x0, coef_sat             // x0 = endereco dos
coeficientes do caso saturado
    mov x1, #2                   // x1 = numero de
coeficientes do caso saturado
    mov w2, #131072              // w2 = x = 2.0 em Q16.16
    bl poly_horner_q16_sat        // avalia o polinomio
esperando saturacao
after_saturation_case:
    adr x9, result_sat           // x9 = endereco para salvar
o resultado saturado
    str w0, [x9]                 // grava o valor retornado
    adr x9, status_sat           // x9 = endereco para salvar
o status do caso saturado
    str w1, [x9]                 // grava o status retornado

    adr x0, coef_ok              // x0 = coeficientes validos
para o teste N=0
    mov x1, xzr                  // x1 = N=0 para disparar
erro especifico
    mov w2, #262144              // w2 = x de teste
    bl poly_horner_q16_sat        // chama a rotina esperando
status 2

```

```

after_bad_n_case:
    adr x9, result_bad_n           // x9 = endereco para salvar
o resultado do caso N invalido
    str w0, [x9]                   // grava o valor retornado
    adr x9, status_bad_n           // x9 = endereco para salvar
o status do caso N invalido
    str w1, [x9]                   // grava o status retornado

    mov x0, xzr                    // x0 = ponteiro nulo para
testar erro de entrada
    mov x1, #3                     // x1 = numero de
coeficientes valido
    mov w2, #262144                // w2 = x de teste
    bl poly_horner_q16_sat          // chama a rotina esperando
status 1
after_null_case:
    adr x9, result_null            // x9 = endereco para salvar
o resultado do caso nulo
    str w0, [x9]                   // grava o valor retornado
    adr x9, status_null            // x9 = endereco para salvar
o status do caso nulo
    str w1, [x9]                   // grava o status retornado

    adr x9, result_ok              // x9 = endereco do
resultado do caso simples
    ldr w10, [x9]                  // carrega o valor calculado
    mov w11, #2949120              // w11 = 45.0 em Q16.16,
valor esperado
    cmp w10, w11                   // compara o resultado com o
esperado
    b.ne .fail                     // falha se o valor estiver
incorreto

    adr x9, status_ok              // x9 = endereco do status
do caso simples
    ldr w10, [x9]                  // carrega o status
retornado
    cbnz w10, .fail                // falha se o status nao for
zero

```



```

    adr x9, result_sat                // x9 = endereco do
resultado do caso saturado
    ldr w10, [x9]                     // carrega o valor calculado
    mov w11, #0x7fffffff              // w11 = limite maximo
esperado apos saturacao
    cmp w10, w11                      // compara o resultado com o
limite esperado
    b.ne .fail                        // falha se o valor estiver
incorreto

    adr x9, status_sat                // x9 = endereco do status
do caso saturado
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #3                       // verifica se a saturacao
foi sinalizada
    b.ne .fail                        // falha se o status estiver
incorreto

    adr x9, result_bad_n              // x9 = endereco do
resultado do caso N invalido
    ldr w10, [x9]                     // carrega o valor retornado
    cbnz w10, .fail                  // falha se o retorno nao
for zero

    adr x9, status_bad_n              // x9 = endereco do status
do caso N invalido
    ldr w10, [x9]                     // carrega o status
retornado
    cmp w10, #2                       // verifica se o codigo de
erro foi 2
    b.ne .fail                        // falha se o status estiver
incorreto

    adr x9, result_null              // x9 = endereco do
resultado do caso nulo
    ldr w10, [x9]                     // carrega o valor retornado
    cbnz w10, .fail                  // falha se o retorno nao
for zero

```

```

    adr x9, status_null           // x9 = endereço do status
do caso nulo
    ldr w10, [x9]                 // carrega o status
retornado
    cmp w10, #1                   // verifica se o código de
erro foi 1
    b.ne .fail                    // falha se o status estiver
incorreto

    mov w0, wzr                   // w0 = exit code 0 para
sucesso total
    b .exit                       // encerra o programa com
sucesso

.fail:
    mov w0, #1                    // w0 = exit code 1 para
falha em alguma validacao

.exit:
    mov x8, #93                   // x8 = syscall exit do
Linux AArch64
    svc #0                        // encerra o processo com o
código em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global poly_horner_q16_sat
.type poly_horner_q16_sat, %function

poly_horner_q16_sat:
    cbz x0, .null_ptr             // retorna erro se o
ponteiro de coeficientes for nulo
    cbz x1, .bad_n                // retorna erro se N for
zero

    mov x9, x1                    // x9 = contador de
coeficientes restantes

```

```

    ldr w3, [x0], #4           // le o primeiro coeficiente
do polinomio
    sxtw x3, w3                // expande o acumulador
inicial para 64 bits com sinal
    sxtw x2, w2                // expande x para 64 bits
com sinal
    mov w8, wzr                // w8 = status de saturacao
inicializado com zero

    mov w4, #0x7fffffff        // w4 = limite maximo de 32
bits com sinal
    sxtw x4, w4                // expande o limite maximo
para 64 bits
    mov w5, #0x80000000        // w5 = limite minimo de 32
bits com sinal
    sxtw x5, w5                // expande o limite minimo
para 64 bits

    subs x9, x9, #1           // desconta o coeficiente
inicial ja carregado
    b.eq .done                 // encerra se o polinomio
tiver apenas um coeficiente

.loop:
    ldr w6, [x0], #4           // le o proximo coeficiente
e avanca o ponteiro
    sxtw x6, w6                // expande o coeficiente
para 64 bits com sinal
    mul x7, x3, x2             // calcula acc * x em 64
bits
    asr x7, x7, #16            // normaliza o produto de
volta para Q16.16
    add x7, x7, x6             // soma o coeficiente atual
pelo metodo de Horner

    cmp x7, x4                 // verifica se o acumulador
ultrapassou o limite maximo
    b.gt .sat_max              // satura para o maximo se
houver overflow positivo

```

```

        cmp x7, x5                // verifica se o acumulador
ficou abaixo do limite minimo
        b.lt .sat_min            // satura para o minimo se
houver overflow negativo

        mov x3, x7                // atualiza o acumulador com
o valor calculado
        b .next                 // segue para o proximo
coeficiente

.sat_max:
        mov x3, x4                // fixa o acumulador no
limite maximo
        mov w8, #3                // marca status 3 para
indicar saturacao
        b .next                 // segue para o proximo
coeficiente

.sat_min:
        mov x3, x5                // fixa o acumulador no
limite minimo
        mov w8, #3                // marca status 3 para
indicar saturacao

.next:
        subs x9, x9, #1           // decrementa a quantidade
de coeficientes restantes
        b.ne .loop               // continua enquanto ainda
houver coeficientes a processar

.done:
        mov w0, w3                // retorna o valor final do
polinomio em w0
        mov w1, w8                // retorna o status final em
w1
        ret                      // volta ao chamador

.bad_n:
        mov w0, wzr               // retorna valor zero quando
N e invalido

```

```

        mov w1, #2                                // retorna status 2 para N
invalido
        ret                                        // volta ao chamador

.null_ptr:
        mov w0, wzr                                // retorna valor zero quando
o ponteiro e nulo
        mov w1, #1                                // retorna status 1 para
ponteiro nulo
        ret                                        // volta ao chamador

.section .note.GNU-stack,"",@progbits

```

12.0

```
renato@NOTE-PESSOAL: ~/fa X + v
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q12$ qemu-aarch64 -g 2401 ./q12 &
[1] 12206
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q12$ gdb-multiarch -q ./q12
Reading symbols from ./q12...
(gdb) target remote :2401
Remote debugging using :2401
_start () at main.S:45
45      mov w19, wzr                                // w19 = flags acumuladas do pipeline inicializadas com zero
(gdb) break after_normalize
Breakpoint 1 at 0x40013c: file main.S, line 53.
(gdb) break after_dot
Breakpoint 2 at 0x4001c4: file main.S, line 91.
(gdb) break after_sqrt
Breakpoint 3 at 0x400230: file main.S, line 128.
(gdb) break after_classify
Breakpoint 4 at 0x400264: file main.S, line 144.
(gdb) c
Continuing.

Breakpoint 1, after_normalize () at main.S:53
53      adr x9, normalize_status                    // x9 = endereco para salvar o status da normalizacao
(gdb) print/u $w0
$1 = 0
(gdb) x/4wd &normalized_i32
0x400510: 8388607 -8388608 500000 -500000
(gdb) c
Continuing.

Breakpoint 2, after_dot () at main.S:91
91      adr x9, energy_value                        // x9 = endereco para salvar a energia calculada
(gdb) print/u $w1
$2 = 0
(gdb) print/d $x0
$3 = 141237471578113
(gdb) c
Continuing.

Breakpoint 3, after_sqrt () at main.S:128
128     adr x9, magnitude_value                    // x9 = endereco para salvar a magnitude calculada

Breakpoint 3, after_sqrt () at main.S:128
128     adr x9, magnitude_value                    // x9 = endereco para salvar a magnitude calculada
(gdb) print/u $w1
$4 = 0
(gdb) print (double)$d0
$5 = 5942168.6188232657
(gdb) c
Continuing.

Breakpoint 4, after_classify () at main.S:144
144     adr x9, final_code                         // x9 = endereco para salvar o codigo final
(gdb) print/u $w0
$6 = 2
(gdb) c
Continuing.
[Inferior 1 (process 1) exited with code 02]
(gdb) q
[1]+  Exit 2                  qemu-aarch64 -g 2401 ./q12
renato@NOTE-PESSOAL:~/faculdade/26.1-sistemasembarcados/drs/dr2/at/q12$ |
```

main.s

```
.data
samples_i16:
    .hword 32767, -32768, 1000, -1000
.balign 4
normalized_i32:
    .word 0, 0, 0, 0
expected_normalized:
    .word 8388607, -8388608, 500000, -500000
.balign 8
energy_value:
    .quad 0
expected_energy:
    .quad 141237471578113
mean_value:
    .quad 0
```

```

expected_mean:
    .quad 35309367894528
.balign 8
magnitude_value:
    .double 0.0
expected_magnitude:
    .double 5942168.618823266
magnitude_tol:
    .double 0.001
threshold_value:
    .double 1000000.0
.balign 4
normalize_status:
    .word 0
dot_status:
    .word 0
div_status:
    .word 0
sqrt_status:
    .word 0
flags_value:
    .word 0
final_code:
    .word 0

.text
.global _start

_start:
    mov w19, wzr                // w19 = flags acumuladas do
pipeline inicializadas com zero

    adr x0, samples_i16         // x0 = endereco das
amostras int16 de entrada
    adr x1, normalized_i32      // x1 = endereco do vetor
normalizado de saida
    mov x2, #4                  // x2 = quantidade de
amostras do pipeline
    mov w3, #500                // w3 = ganho de
normalizacao

```

```

        bl normalize_saturate_i16_to_i32           // normaliza o vetor de
entrada para int32 com saturacao
after_normalize:
        adr x9, normalize_status                 // x9 = endereco para salvar
o status da normalizacao
        str w0, [x9]                             // grava o status retornado
pela rotina
        cmp w0, #2                               // verifica se a falha foi
de ponteiro nulo
        b.ne .check_norm_n                      // segue para o proximo
teste se nao for esse caso
        orr w19, w19, #1                         // liga o bit 0 das flags
para ponteiro nulo
        b .scan_saturation                      // continua o fluxo para
registrar outras condicoes

.check_norm_n:
        cmp w0, #1                               // verifica se a falha foi N
invalido
        b.ne .scan_saturation                  // segue se o status nao for
1
        orr w19, w19, #2                         // liga o bit 1 das flags
para N invalido

.scan_saturation:
        adr x10, normalized_i32                 // x10 = ponteiro para
varrer o vetor normalizado
        mov x11, #4                             // x11 = contador de
elementos a inspecionar
        mov w12, #8388607                       // w12 = limite maximo de
saturacao
        mov w13, #0xFF800000                   // w13 = limite minimo de
saturacao em 32 bits

.scan_loop:
        ldr w14, [x10], #4                      // le um valor normalizado e
avanca o ponteiro
        cmp w14, w12                            // verifica se o valor bate
no limite maximo

```



```

        b.eq .mark_fault           // marca fault se houve
saturacao positiva
        cmp w14, w13              // verifica se o valor bate
no limite minimo
        b.eq .mark_fault           // marca fault se houve
saturacao negativa
        b .scan_next              // segue para o proximo
elemento se nao houve saturacao

.mark_fault:
        orr w19, w19, #8           // liga o bit 3 das flags
para saturacao ou overflow relevante

.scan_next:
        subs x11, x11, #1          // decrementa o contador de
elementos restantes
        b.ne .scan_loop           // continua a varredura
enquanto houver elementos

        adr x0, normalized_i32     // x0 = vetor A do produto
escalar
        adr x1, normalized_i32     // x1 = vetor B do produto
escalar, igual ao proprio vetor
        mov x2, #4                 // x2 = numero de elementos
do vetor
        bl dot_i32_checked         // calcula a energia
aproximada via dot(v, v)
after_dot:
        adr x9, energy_value       // x9 = endereco para salvar
a energia calculada
        str x0, [x9]              // grava a soma acumulada
retornada
        adr x9, dot_status         // x9 = endereco para salvar
o status do dot product
        str w1, [x9]              // grava o status retornado
        cmp w1, #2                 // verifica se houve erro de
ponteiro nulo
        b.ne .check_dot_n         // segue para o proximo
teste se nao for esse caso

```

```

    orr w19, w19, #1                // liga o bit 0 das flags
para ponteiro nulo
    b .divide_energy                // continua o pipeline

.check_dot_n:
    cmp w1, #1                      // verifica se houve erro de
N invalido
    b.ne .check_dot_overflow        // segue se o status nao for
1
    orr w19, w19, #2                // liga o bit 1 das flags
para N invalido
    b .divide_energy                // continua o pipeline

.check_dot_overflow:
    cmp w1, #3                      // verifica se houve
overflow no acumulador
    b.ne .divide_energy            // segue se nao houve
overflow
    orr w19, w19, #3                // liga o bit 3 das flags
para overflow relevante

.divide_energy:
    mov x1, #4                      // x1 = denominador N para
calcular a media da energia
    bl safe_div_s64                 // divide a energia por N
com politica de erro segura
after_divide:
    adr x9, mean_value              // x9 = endereco para salvar
a media calculada
    str x0, [x9]                    // grava o valor retornado
pela divisao segura
    adr x9, div_status              // x9 = endereco para salvar
o status da divisao
    str w1, [x9]                    // grava o status retornado
    cmp w1, wzr                     // verifica se a divisao foi
bem sucedida
    b.eq .compute_sqrt              // segue para a raiz
quadrada se nao houve erro
    orr w19, w19, #2                // liga o bit 2 das flags
para erro numerico/divisao

```

```

.compute_sqrt:
    scvtf d0, x0                // converte a media inteira
para double
    mov w0, #40                 // w0 = numero de iteracoes
para Newton-Raphson
    bl sqrt_nr                  // calcula a magnitude a
partir da media da energia
after_sqrt:
    adr x9, magnitude_value     // x9 = endereco para salvar
a magnitude calculada
    str d0, [x9]                // grava o valor retornado
pela raiz quadrada
    adr x9, sqrt_status         // x9 = endereco para salvar
o status da raiz quadrada
    str w1, [x9]                // grava o status retornado
    cmp w1, wzr                 // verifica se a raiz
quadrada foi bem sucedida
    b.eq .classify              // segue para a
classificacao se nao houve erro
    orr w19, w19, #4            // liga o bit 2 das flags
para erro numerico

.classify:
    adr x9, flags_value         // x9 = endereco para salvar
a bitmask de flags
    str w19, [x9]               // grava as flags acumuladas
do pipeline
    mov w0, w19                 // w0 = flags de erro para a
classificacao final
    adr x10, threshold_value    // x10 = endereco do limiar
de magnitude
    ldr d1, [x10]               // d1 = limiar usado na
classificacao
    bl classify_health           // calcula o codigo final de
diagnostico
after_classify:
    adr x9, final_code          // x9 = endereco para salvar
o codigo final

```

```

        str w0, [x9]                // grava o código retornado
pela classificacao
        mov w20, w0                // preserva o código final
em w20 para o encerramento

        adr x10, normalized_i32    // x10 = endereço do vetor
normalizado produzido
        adr x11, expected_normalized // x11 = endereço do vetor
normalizado esperado
        mov x12, #4                // x12 = contador de
elementos do vetor

.check_norm_loop:
        ldr w13, [x10], #4         // lê um valor normalizado
produzido
        ldr w14, [x11], #4         // lê o valor esperado
correspondente
        cmp w13, w14               // compara a saída gerada
com a esperada
        b.ne .validation_fail      // falha se houver
divergencia
        subs x12, x12, #1           // decrementa o contador de
elementos restantes
        b.ne .check_norm_loop      // continua enquanto ainda
houver elementos a validar

        adr x9, normalize_status   // x9 = endereço do status
da normalizacao
        ldr w10, [x9]              // carrega o status
retornado
        cbnz w10, .validation_fail // falha se a normalizacao
nao terminou com status 0

        adr x9, dot_status         // x9 = endereço do status
do dot product
        ldr w10, [x9]              // carrega o status
retornado
        cbnz w10, .validation_fail // falha se o dot product
nao terminou com status 0

```

```

    adr x9, energy_value           // x9 = endereço da energia
calculada
    ldr x10, [x9]                 // carrega a energia
produzida
    adr x11, expected_energy      // x11 = endereço da energia
esperada
    ldr x12, [x11]                // carrega a energia
esperada
    cmp x10, x12                  // compara energia produzida
com energia esperada
    b.ne .validation_fail        // falha se os valores
divergirem

    adr x9, div_status            // x9 = endereço do status
da divisao
    ldr w10, [x9]                 // carrega o status
retornado
    cbnz w10, .validation_fail    // falha se a divisao nao
terminou com status 0

    adr x9, mean_value           // x9 = endereço da media
calculada
    ldr x10, [x9]                 // carrega a media produzida
    adr x11, expected_mean       // x11 = endereço da media
esperada
    ldr x12, [x11]                // carrega a media esperada
    cmp x10, x12                  // compara media produzida
com media esperada
    b.ne .validation_fail        // falha se os valores
divergirem

    adr x9, sqrt_status          // x9 = endereço do status
da raiz quadrada
    ldr w10, [x9]                 // carrega o status
retornado
    cbnz w10, .validation_fail    // falha se a raiz quadrada
nao terminou com status 0

    adr x9, magnitude_value      // x9 = endereço da
magnitude calculada

```

```

    ldr d0, [x9] // carrega a magnitude
produzida
    adr x10, expected_magnitude // x10 = endereco da
magnitude esperada
    ldr d1, [x10] // carrega a magnitude
esperada
    fsub d2, d0, d1 // calcula o erro numerico
da magnitude
    fabs d2, d2 // toma o valor absoluto do
erro
    adr x11, magnitude_tol // x11 = endereco da
tolerancia da magnitude
    ldr d3, [x11] // carrega a tolerancia
numerica
    fcmp d2, d3 // compara o erro com a
tolerancia
    b.gt .validation_fail // falha se o erro for maior
que o permitido

    adr x9, flags_value // x9 = endereco da bitmask
de flags gravada
    ldr w10, [x9] // carrega as flags finais
    cmp w10, #8 // verifica se apenas o bit
de saturacao ficou ativo
    b.ne .validation_fail // falha se a bitmask
divergir do esperado

    cmp w20, #2 // verifica se o codigo
final de diagnostico foi FAULT
    b.ne .validation_fail // falha se o codigo final
estiver incorreto

    mov w0, w20 // w0 = codigo final do
pipeline para o exit code
    b .exit // encerra o programa com o
diagnostico correto

.validation_fail:
    mov w0, #255 // w0 = codigo especial para
sinalizar falha interna na validacao

```

```

.exit:
    mov x8, #93                // x8 = syscall exit do
Linux AArch64
    svc #0                    // encerra o processo com o
codigo em w0

.section .note.GNU-stack,"",@progbits

```

lib.s

```

.text
.global normalize_saturate_i16_to_i32
.global dot_i32_checked
.global safe_div_s64
.global sqrt_nr
.global classify_health

.type normalize_saturate_i16_to_i32, %function
normalize_saturate_i16_to_i32:
    cbz x0, .norm_null_ptr    // retorna erro se o
ponteiro de entrada for nulo
    cbz x1, .norm_null_ptr    // retorna erro se o
ponteiro de saida for nulo
    cbz x2, .norm_zero_len    // retorna erro se N for
zero

    mov x4, #-8388608         // x4 = limite minimo de
saturacao
    mov x5, #8388607         // x5 = limite maximo de
saturacao

.norm_loop:
    ldrsh w6, [x0], #2        // le uma amostra int16 com
extensao de sinal
    smull x7, w6, w3          // multiplica a amostra pelo
ganho em 64 bits
    cmp x7, x4               // compara com o limite
minimo

```

```

        b.lt .norm_sat_min           // satura para o minimo se
estiver abaixo do intervalo
        cmp x7, x5                   // compara com o limite
maximo
        b.gt .norm_sat_max           // satura para o maximo se
estiver acima do intervalo
        str w7, [x1], #4             // grava o valor sem
saturacao na saida
        b .norm_next                 // segue para a proxima
amostra

.norm_sat_min:
        str w4, [x1], #4             // grava o valor minimo
saturado
        b .norm_next                 // segue para a proxima
amostra

.norm_sat_max:
        str w5, [x1], #4             // grava o valor maximo
saturado

.norm_next:
        subs x2, x2, #1              // decrementa o numero de
amostras restantes
        b.ne .norm_loop              // continua enquanto ainda
houver amostras

        mov w0, wzr                  // retorna status 0 para
sucesso
        ret                          // volta ao chamador

.norm_zero_len:
        mov w0, #1                   // retorna status 1 para N=0
        ret                          // volta ao chamador

.norm_null_ptr:
        mov w0, #2                   // retorna status 2 para
ponteiro nulo
        ret                          // volta ao chamador

```



```

.type dot_i32_checked, %function
dot_i32_checked:
    cbz x0, .dot_null_ptr           // retorna erro se o vetor A
for nulo
    cbz x1, .dot_null_ptr           // retorna erro se o vetor B
for nulo
    cbz x2, .dot_zero_len           // retorna erro se N for
zero

    mov x3, xzr                     // x3 = acumulador inicial
da soma em 64 bits

.dot_loop:
    ldr w4, [x0], #4                // le o proximo elemento de
A
    ldr w5, [x1], #4                // le o proximo elemento de
B
    smull x6, w4, w5                // calcula o produto em 64
bits
    adds x7, x3, x6                 // soma o produto ao
acumulador atualizando flags
    b.vs .dot_overflow              // interrompe se o
acumulador sofrer overflow
    mov x3, x7                      // atualiza o acumulador com
a nova soma
    subs x2, x2, #1                 // decrementa a quantidade
de elementos restantes
    b.ne .dot_loop                  // continua enquanto ainda
houver elementos

    mov x0, x3                      // retorna a soma final em
x0
    mov w1, wzr                     // retorna status 0 para
sucesso
    ret                             // volta ao chamador

.dot_overflow:
    mov x0, x3                      // retorna a soma acumulada
antes do overflow

```

```

        mov w1, #3                                // retorna status 3 para
overflow detectado
        ret                                        // volta ao chamador

.dot_zero_len:
        mov x0, xzr                                // retorna soma zero quando
N e invalido
        mov w1, #1                                // retorna status 1 para N=0
        ret                                        // volta ao chamador

.dot_null_ptr:
        mov x0, xzr                                // retorna soma zero quando
ha ponteiro nulo
        mov w1, #2                                // retorna status 2 para
ponteiro nulo
        ret                                        // volta ao chamador

.type safe_div_s64, %function
safe_div_s64:
        cbz x1, .div_zero                          // trata imediatamente o
caso de divisor zero

        mov x2, #1                                // prepara a constante base
        lsl x2, x2, #63                            // x2 = INT64_MIN
        cmp x0, x2                                // verifica se o numerador e
INT64_MIN
        b.ne .do_div                               // segue para a divisao
normal se nao for o caso especial
        cmp x1, #-1                                // verifica se o denominador
e -1
        b.ne .do_div                               // segue para a divisao
normal se nao for o caso especial

        mov x0, #-1                                // monta um valor com todos
os bits em 1
        lsr x0, x0, #1                             // x0 =
0x7FFF_FFFF_FFFF_FFFF
        mov w1, #2                                // retorna status 2 para
overflow evitado
        ret                                        // volta ao chamador

```

```

.do_div:
    sdiv x0, x0, x1                // faz a divisao assinada no
caso normal
    mov w1, wzr                    // retorna status 0 para
sucesso
    ret                            // volta ao chamador

.div_zero:
    mov x0, #-1                    // monta um valor com todos
os bits em 1
    lsr x0, x0, #1                 // x0 =
0x7FFF_FFFF_FFFF_FFFF
    mov w1, #1                     // retorna status 1 para
divisor zero
    ret                            // volta ao chamador

.type sqrt_nr, %function
sqrt_nr:
    fmov d2, xzr                    // d2 = 0.0 para as
validacoes iniciais
    fcmp d0, d2                    // verifica se a entrada e
negativa
    b.lt .sqrt_bad_input           // sai com erro se a < 0

    cbz w0, .sqrt_bad_iter         // sai com erro se max_iter
for zero

    fmov d4, d0                    // d4 = copia de 'a' para a
recorrencia
    fmov d3, #1.0                  // d3 = constante 1.0 para o
chute minimo
    fcmp d0, d3                    // verifica se a >= 1.0
    b.ge .sqrt_use_a               // usa a propria entrada
como chute se a >= 1

    fmov d2, d3                    // d2 = chute inicial 1.0
    b .sqrt_loop_setup             // vai para a preparacao do
laco

```

```

.sqrt_use_a:
    fmov d2, d0                // d2 = chute inicial igual
a 'a'

.sqrt_loop_setup:
    mov w2, w0                 // w2 = contador de
iteracoes restantes
    fmov d5, #0.5              // d5 = constante 0.5 usada
no passo de Newton

.sqrt_loop:
    fdiv d6, d4, d2            // d6 = a / x_n
    fadd d2, d2, d6            // soma x_n + a/x_n
    fmul d2, d2, d5            // calcula o proximo chute
do metodo
    subs w2, w2, #1            // decrementa o numero de
iteracoes restantes
    b.ne .sqrt_loop           // continua ate completar
max_iter passos

    fmov d0, d2                // retorna a aproximacao
final em d0
    mov w1, wzr                // retorna status 0 para
sucesso
    ret                        // volta ao chamador

.sqrt_bad_input:
    fmov d0, xzr               // retorna 0.0 em caso de
entrada invalida
    mov w1, #1                 // retorna status 1 para a <
0
    ret                        // volta ao chamador

.sqrt_bad_iter:
    fmov d0, xzr               // retorna 0.0 em caso de
max_iter invalido
    mov w1, #2                 // retorna status 2 para
max_iter igual a zero
    ret                        // volta ao chamador

```

```

.type classify_health, %function
classify_health:
    mov w2, wzr                // w2 = classe provisoria
    inicializada como OK
    cbnz w0, .class_fault      // se houver flags de erro,
    classifica direto como FAULT
    fcmp d0, d1                // compara magnitude com o
    limiar
    b.gt .class_degraded       // desvia para DEGRADED se
    magnitude > limiar
    b .dispatch                // segue para o despacho
    final mantendo OK

.class_fault:
    mov w2, #2                // marca a classe provisoria
    como FAULT
    b .dispatch                // segue para o despacho
    final

.class_degraded:
    mov w2, #1                // marca a classe provisoria
    como DEGRADED

.dispatch:
    cmp w2, #0                // verifica se a classe
    final e OK
    b.eq .ret_ok              // retorna OK se w2 == 0
    cmp w2, #1                // verifica se a classe
    final e DEGRADED
    b.eq .ret_degraded        // retorna DEGRADED se w2 ==
    1
    mov w0, #2                // retorna FAULT no caso
    restante
    ret                        // volta ao chamador

.ret_ok:
    mov w0, wzr                // retorna codigo 0 para OK
    ret                        // volta ao chamador

.ret_degraded:

```

```
    mov w0, #1                                // retorna codigo 1 para
DEGRADED
    ret                                        // volta ao chamador

.section .note.GNU-stack,"",@progbits
```